

Budapest University of Technology and Economics
Faculty of Electrical Engineering and Informatics
Department of Automation and Applied Informatics

Comparison of Reinforcement Learning and Model Predictive Control for a Drone Equipped with a Manipulator

Scientific Students' Association Report

Author:

Tamás Epres

Advisor:

Péter Antal, PhD (HUN-REN SZTAKI)
Bence Szinyéri, (BME AUT)

2026

Contents

Kivonat i

Abstract ii

1 Introduction 1

2 System Modeling 3

2.1 Quadcopter Model . . . 3

2.2 Applied Manipulator . . . 4

2.3 Task Description . . . 6

3 Reinforcement Learning 7

3.1 Reinforcement Learning Formulation . . . 7

3.1.1 Observation . . . 8

3.1.2 Action . . . 9

3.1.3 Reward . . . 9

3.1.4 Episode . . . 10

3.1.5 Flat Agent . . . 10

3.2 Curriculum Learning . . . 11

3.2.1 Formulation . . . 11

3.2.2 Training with and without CL . . . 11

3.3 Hierarchical Reinforcement Learning . . . 13

3.3.1 Formulation . . . 13

3.3.2 Low-Level Observation . . . 14

3.3.3 Low-Level Action . . . 15

3.3.4 Low-Level Reward . . . 15

3.3.5 Low-Level Episode . . . 15

3.3.6 Low-Level Agent Results . . . 15

3.3.7 High-Level Policy . . . 19

3.3.8 High-Level Agent Results . . . 19

4 Model Predictive Control 21

4.1 Model Predictive Control Formulation . . . 21

4.1.1 State and Input . . . 22

4.1.2 Prediction Model . . . 22
4.1.3 Optimal Control Problem . . . 22
4.2 System Identification . . . 24
4.2.1 Model Structure . . . 24
4.2.2 Training . . . 25
4.2.3 Validation of the Identification . . . 26

5 Safety Filter **29**
5.1 Formulation . . . 29
5.1.1 Integration into the Hierarchy . . . 29
5.1.2 Safety Filter Problem . . . 31
5.1.3 Constraints . . . 32
5.1.4 Additional Rules . . . 32

6 Comparison **33**
6.1 Evaluation Setup . . . 33
6.2 Results . . . 33
6.3 Rescue of the RL Failures . . . 34

7 Conclusion and Future Work **36**

Acknowledgements **37**

Bibliography **38**

Appendix **41**
A.1 Hyperparameters . . . 41
A.2 Technical Details . . . 41

Kivonat

Az autonóm légi járművek alkalmazása mára szinte mindennapossá vált. Érdekes és ígéretes kutatási irány annak vizsgálata, hogyan használhatók ezek a járművek olyan feladatokra, amelyekben a drón fizikai kapcsolatba lép a környezetével, és kölcsönhatásba kerül vele.

A dolgozat egy ilyen rendszert vizsgál: egy Bumblebee-alapú kvadkopter aljára spirálrobotok által ihletett, egyszerűsített, sok szabadságfokú, síkban mozgó manipulátor került. A szabályozó feladata olyan trajektória előállítása, amelynek során a drón egy előre nem ismert fizikai paraméterekkel (mérettel és tömeggel) rendelkező csomagot megfog, majd a lehető leggyorsabban a célpontba szállít. A rendszer vizsgálata MuJoCo-alapú szimulációs környezetben történik.

A dolgozatban két módszer került implementálásra. Az első adatalapú: egy Proximal Policy Optimization (PPO) alapú, hierarchikus megerősítéses tanulási (HRL) architektúra, amely nem ismeri a rendszer modelljét, hanem a szimulációban gyűjtött tapasztalatokból tanul. Ebben egy magas szintű ágens sebességreferenciát generál, amelyet egy előzetesen betanított, alacsony szintű ágens követ. A tanítást curriculum learning (CL) segíti, amelynek során a feladat nehézsége és a csomag paramétereinek változatossága fokozatosan nő.

A másik megoldás a modellprediktív szabályozás (MPC), amely a magas szintű ágens helyét veszi át, és az alacsony szintű ágensnek ad sebességreferenciát. Ehhez a drón és az azt irányító alacsony szintű ágens együttes viselkedését szimulációs adatokból egy reziduális neurális hálózattal (ResNet) identifikáltuk, és az MPC ezt a modellt használja predikcióra.

Végül a dolgozat javaslatot tesz egy hibrid módszerre, amely ötvözi a két megközelítés erősségeit: egy prediktív biztonsági szűrő az identifikált modell segítségével ellenőrzi az RL-ágens beavatkozójeleit, és csak akkor avatkozik be, ha azok veszélyes állapotba vinnék a drónt. A három módszert 300 azonos, véletlenszerűen generált feladaton hasonlítjuk össze. Az RL-ágens a leggyorsabb, de a feladatoknak csak a 94,3%-át teljesíti, és főként a nehéz csomagoknál hibázik. Az MPC a legmegbízhatóbb (99,7%), viszont lassabb és számításigényesebb. A biztonsági szűrő 97,7%-ra növeli az RL-ágens sikerességét, miközben megőrzi a gyorsaságát.

Abstract

The use of autonomous aerial vehicles has become almost commonplace. An interesting and promising research direction is to investigate how these vehicles can be used for tasks in which the drone makes physical contact with its environment and interacts with it.

This thesis studies such a system: a simplified, planar manipulator with many degrees of freedom, inspired by spiral robots, is mounted under a Bumblebee-based quadcopter. The task of the controller is to generate a trajectory along which the drone grasps a payload with unknown physical parameters (size and mass) and transports it to a goal position as fast as possible. The system is studied in a MuJoCo-based simulation environment.

Two methods are implemented in the thesis. The first one is data-driven: a hierarchical reinforcement learning (HRL) architecture based on Proximal Policy Optimization (PPO), which does not know the model of the system, but learns from the experience collected in simulation. In this architecture, a high-level agent generates a velocity reference, which is tracked by a pre-trained low-level agent. Training is supported by curriculum learning (CL), during which the difficulty of the task and the variability of the payload parameters are gradually increased.

The other solution is model predictive control (MPC), which takes the place of the high-level agent and provides the velocity reference to the low-level agent. To this end, the combined behavior of the drone and the low-level agent controlling it was identified from simulation data with a residual neural network (ResNet), which the MPC uses for prediction.

Finally, the thesis proposes a hybrid method that combines the strengths of the two approaches: a predictive safety filter checks the commands of the RL agent with the identified model, and it intervenes only if they would drive the drone into an unsafe state. The three methods are compared on the same 300 randomly generated tasks. The RL agent is the fastest, but it completes only 94.3 % of the tasks, failing mainly with heavy payloads. The MPC is the most reliable (99.7 %), but it is slower and computationally more demanding. The safety filter raises the success rate of the RL agent to 97.7 %, while it preserves its speed.

Chapter 1

Introduction

The applications of unmanned aerial vehicles (UAVs) are solving increasingly complex tasks. Today, their use is no longer limited to passive tasks that require no physical contact with the environment, such as surveillance, video recording, or security and patrol missions. UAVs are now also deployed in tasks where the drone interacts physically with its surroundings. Examples include package delivery from a moving platform [3], the retrieval of flying vehicles [23], pipeline crack sealing [12], and even the collection of endangered plants from dangerous cliffs [10].

This thesis is motivated by the ever faster and steadily growing demand for transportation. Supply chains require increasingly fast and inexpensive delivery, while safe transport and the protection of human life and property remain of paramount importance. An aerial manipulation system of this kind can offer a solution, as the drones appearing on the market are becoming cheaper and faster. Drone delivery is no longer only a research topic: large logistics companies are already introducing it, e.g., Amazon plans to extend its package delivery services with the drones of its Prime Air service, shown in Figure 1.1. In such services, the drone carries the package to the customer autonomously, which is the kind of transportation task that this thesis addresses with a manipulator.



Figure 1.1: Delivery drones of Amazon Prime Air, with which Amazon plans to extend its package delivery services: in flight (left)[1] and on the ground (right)[2].

To interact physically with its surroundings, the drone is equipped with some kind of manipulator, which can range from a simple hook [3], through a multi-degree-of-freedom robotic arm [12], to a deformable, so-called soft robotic manipulator containing compliant segments [7]. This thesis deals with a drone system in which a manipulator inspired by spiral robots [25] is mounted on the underside of the drone. The drone–manipulator system is described in detail in the *System Modeling* chapter. The task of the drone is to pick up a randomly placed package, whose mass and the radius of the cylinder used for grasping are not known in advance, and to deliver it to a

[1]Source: `https://www.aboutamazon.com/news/transportation/amazon-prime-air-drone-delivery-expansion`
[2]Source: `https://www.aboutamazon.com/news/operations/mk30-drone-amazon-delivery-packages`

target location. Throughout the task, safe flight and the maintenance of the yaw orientation are imposed as constraints, and the task must be completed as fast as possible.

Numerous approaches can be applied to realize such a trajectory planning algorithm. In recent years, learning-based (data-driven) control algorithms have become increasingly widespread, and their application has also extended to drone-mounted manipulator tasks [26]. These methods typically do not require an accurate description of the system; instead, they predict the system's behavior based on collected data. They are generally robust to modeling errors and have very short execution times. On the other hand, their training is not trivial and is often a lengthy process, and safe control cannot be guaranteed. A reinforcement learning (RL) algorithm of this kind has been implemented in this thesis as well. To facilitate training, hierarchical RL was applied in combination with curriculum learning. The method is presented in detail in the *Reinforcement Learning* chapter.

Another approach is model predictive control (MPC). In this framework, a representation describing the system is available, based on which an optimization problem can be formulated; solving it yields a suitable sequence of control inputs. This control strategy also has its advantages and disadvantages. Its advantage is that it can provide well-defined control inputs even in situations the drone has not previously encountered, while satisfying certain constraints. Its disadvantages are that, for higher-dimensional problems, its computation time may be longer, and it is sensitive to potential modeling errors. Such an MPC method has been applied in this thesis and is presented in the *Model Predictive Control* chapter, which also describes the system identification procedure used.

Finally, this thesis proposes a solution that combines the two methods so that each compensates for the shortcomings of the other. The two methods are used together in numerous applications [16]; this thesis investigates how MPC can be used to improve the safety of the RL controller. To this end, a safety filter similar to the one in [24] is presented. The filter is an MPC that supervises the control inputs proposed by the RL agent and overrides them whenever they would drive the system into an unsafe state. The method is described in detail in the *Safety Filter* section. Finally, the three approaches are compared in the *Comparison* section.

Chapter 2

System Modeling

This chapter presents the quadcopter, the spiral-robot-inspired manipulator mounted on it [25], and the transportation task to be performed.

2.1 Quadcopter Model

The drone used in this work is the Bumblebee quadcopter of SZTAKI's AIMotionLab. Its modeling and control are described in detail in [2], which the following summary is based on.

The motion of the quadcopter is described in two coordinate frames (Figure 2.1): the inertial frame $\mathcal{F}^i$, fixed to the environment, and the body frame $\mathcal{F}^b$, attached to the center of mass of the drone. The configuration of the drone is given by the pair $(p, R) \in SE(3)$, where $p \in \mathbb{R}^3$ is its position in $\mathcal{F}^i$, also referred to as the world frame, and $R \in SO(3)$ is the rotation matrix from $\mathcal{F}^i$ to $\mathcal{F}^b$. The orientation can equivalently be described by the roll, pitch, and yaw angles $\eta = [\phi,\, \theta,\, \psi]^\top$, with

$$R(\phi, \theta, \psi) = \mathrm{Rot}_x(\phi)\, \mathrm{Rot}_y(\theta)\, \mathrm{Rot}_z(\psi). \tag{2.1}$$

Since Euler angles suffer from singularities, the rotation matrix representation is preferred for control design.

Following [2], the quadcopter is modeled as a rigid body with mass m and inertia matrix J, actuated by four propellers (Figure 2.2). Each propeller $i \in \{1, 2, 3, 4\}$ generates a thrust force and a drag torque proportional to the square of its angular velocity Ω_i:

$$f_i = c\,\Omega_i^2, \qquad \tau_{z,i} = b\,\Omega_i^2, \tag{2.2}$$

where c and b are the thrust and drag coefficients. The collective thrust and the torques in the body frame are

$$\begin{aligned} F &= f_1 + f_2 + f_3 + f_4, \\ \tau_x &= l_x(f_3 + f_4 - f_1 - f_2), \\ \tau_y &= l_y(f_2 + f_3 - f_1 - f_4), \\ \tau_z &= b(\Omega_1^2 + \Omega_3^2 - \Omega_2^2 - \Omega_4^2), \end{aligned} \tag{2.3}$$

where l_x and l_y are the distances of the propellers from the body axes. The dynamics follow the Newton–Euler equations

$$\begin{aligned} m\ddot{p} &= -mge_3 + FRe_3, \\ J\dot{\omega} &= \tau - \omega \times J\omega, \\ \dot{R} &= R\hat{\omega}, \end{aligned} \tag{2.4}$$

where $e_3 = [0,\, 0,\, 1]^\top$, ω is the angular velocity in $\mathcal{F}^b$, and $\hat{\omega}$ is its skew-symmetric matrix. Note that ω is not the time derivative of the Euler angles: the two are related by $\dot{\eta} = W(\eta)\,\omega$, where

$W(\eta)$ is the Euler angle rate matrix. The inputs of the model are the collective thrust F and the torque vector $\tau = [\tau_x, \tau_y, \tau_z]^\top$.

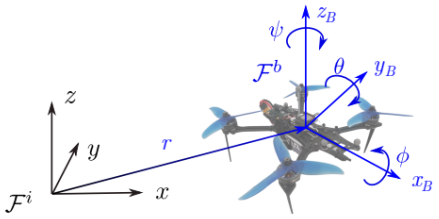


Figure 2.1: Inertial and body frames of the quadcopter, reproduced from [2, Fig. 3].

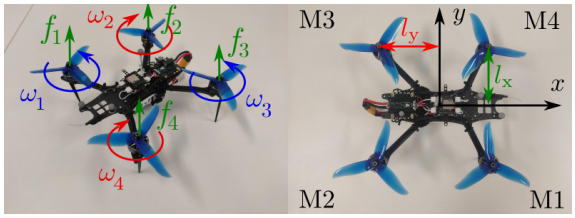


Figure 2.2: Propeller angular velocities and thrusts of the quadcopter, reproduced from [2].

The parameters of the model, measured in [2], are summarized in Table 2.1. The inertia matrix was computed from a detailed CAD model of the drone:

$$J = \begin{bmatrix} 1590.5 & -3.14 & -0.135 \\ -3.14 & 1481.3 & -49.18 \\ -0.135 & -49.18 & 2768.4 \end{bmatrix} \text{kg mm}^2. \tag{2.5}$$

Symbol	Description	Value
m	Mass of the quadcopter	0.605 kg
J	Inertia matrix	see (2.5)
l_x, l_y	Propeller distances from the body axes	0.125 m
c	Thrust coefficient	9.3945×10^{-7} N s^2/rad^2
b	Drag coefficient	N m s^2/rad^2
g	Gravitational acceleration	9.81 m/s^2

Table 2.1: Parameters of the quadcopter model.

2.2 Applied Manipulator

The spiral robot is named after a pattern that occurs frequently in nature, the logarithmic spiral, which can be observed, for example, in the trunk of an elephant or the arm of an octopus. Similarly to these biological examples, the robot can grasp objects of various shapes and masses, such as a tennis ball, an apple, or a pair of headphones [25].

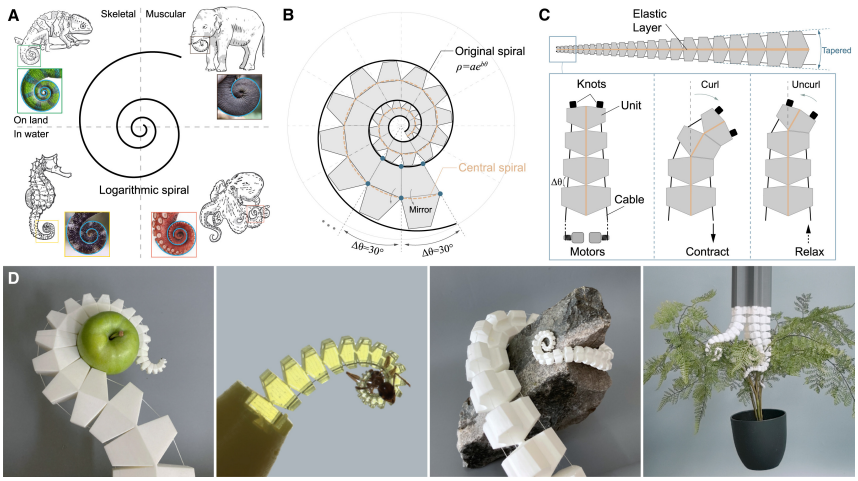


Figure 2.3: Applications and scalability of the spiral robot, reproduced from [25, Fig. 1]. (A) Logarithmic spirals in animal body parts (fitted spirals in blue). (B) Design: the discretized spiral is mirrored to form the robot units. (C) Elastic central layer and motor-driven cables that make the robot curl and uncurl. (D) 3D-printed robots grasping an apple, an ant, a rock, and a plant.

As shown in Figure 2.3(B) and (C), the spiral robot is not a soft manipulator. Instead, it consists of a series of rigid, 3D-printed segments that discretize the logarithmic spiral. The segments are connected by an elastic layer, which pulls the robot back to a straight configuration in its rest state. The robot is actuated by cables, which are routed along the edges of the segments and fixed to the last segment. When one of the cables is pulled, the robot curls in the corresponding direction, as illustrated in Figure 2.3(C).

This thesis can be regarded as an extension of [25]. Although [25] also mounts the spiral robot on a drone (Figure 2.4), it focuses on the fabrication, load-bearing capacity, contact sensing, and scalability of the robot, and uses only a simple control approach, without addressing the modeling or a rigorous control design of the drone–manipulator system.

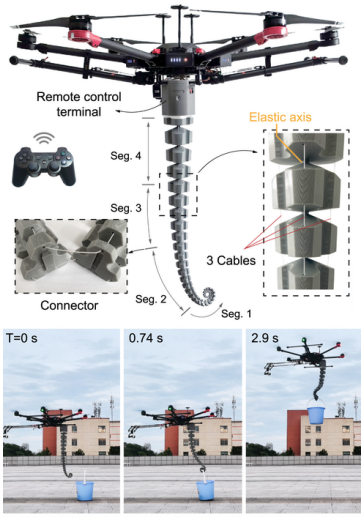


Figure 2.4: The spiral robot mounted on a drone picks up a bucket, reproduced from [25, Fig. 5(E), (F)].

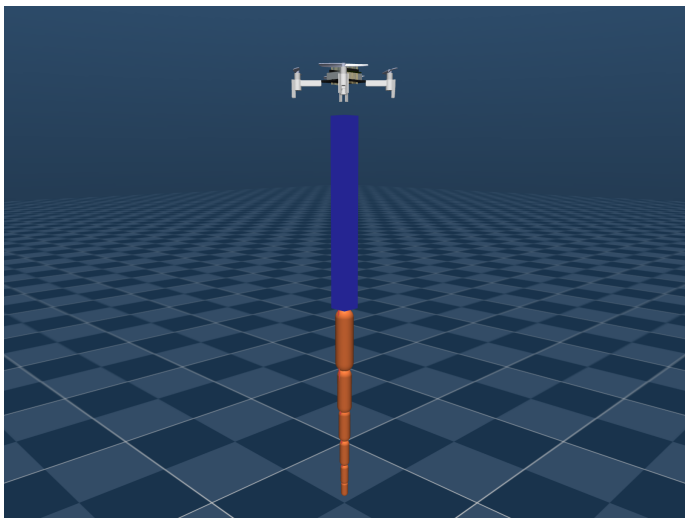


Figure 2.5: MuJoCo model of the applied drone and spiral robot system.

In this thesis, a simplified version of the spiral robot is modeled in the MuJoCo simulation environment [20], as shown in Figure 2.5: the arm consists of seven segments and moves in a single plane.

The dynamic description of such a drone–manipulator system is far from trivial. For systems with many degrees of freedom, directly using the MuJoCo simulation environment within MPC is often a computational bottleneck, since the derivatives of the dynamics required by the optimizer have to be obtained by finite differencing (FD), as noted in [11]. The situation is further complicated by the question of how to describe the system while the drone is carrying a payload with unknown physical parameters. To address this, an identification procedure is presented in detail in the *Model Predictive Control* chapter.

2.3 Task Description

The drone always starts from the same initial position with a random yaw orientation. A payload with unknown physical parameters is placed at a random position, and a goal position is also chosen randomly. The task of the drone is to pick up the payload and then fly to the goal position while carrying it. A typical setup is shown in Figure 2.6.

The payload consists of a cylinder and two cuboids, which are attached to the same side of the cylinder at the two ends of its lateral surface, and held together by a disk, as shown in Figure 2.7.

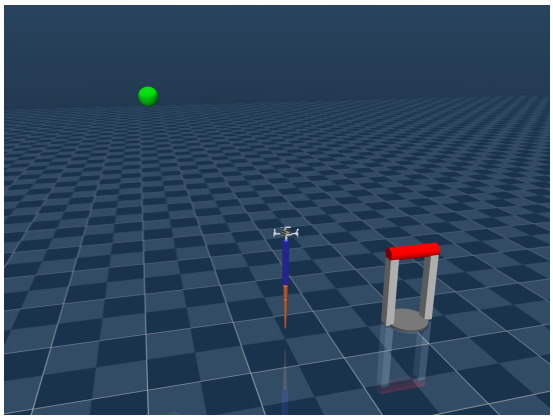


Figure 2.6: A typical scene with a randomly placed payload, a random goal position, and a random initial yaw angle of the drone.

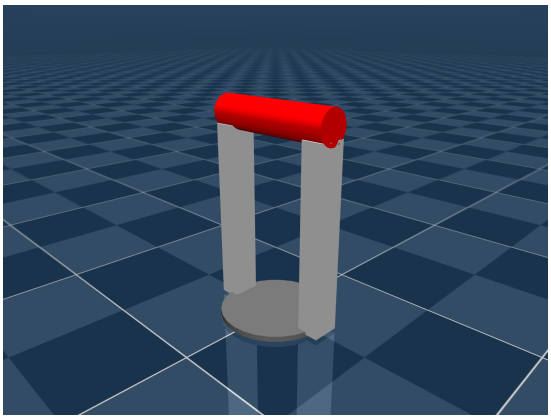


Figure 2.7: Structure of the payload. The robot grasps the red cylinder; its radius and the mass of the disk are not known in advance.

In each task, the radius R_{payload} of the cylinder, the mass m_{payload} of the disk, the height z_{payload} and the horizontal position $(x_{\text{payload}}, y_{\text{payload}})$ of the payload, the horizontal goal position $(x_{\text{goal}}, y_{\text{goal}})$, and the initial yaw angle ψ_0 of the drone are sampled from uniform distributions, whose bounds are summarized in Table 2.2. The height of the goal position is fixed at $z_{\text{goal}} = 1\,\text{m}$.

Parameter	Minimum	Maximum	Unit
Cylinder radius R_{payload}	0.02	0.04	m
Disk mass m_{payload}	0.01	0.25	kg
Payload height z_{payload}	0.45	0.8	m
Payload position x_{payload}, y_{payload}	-1	1	m
Goal position x_{goal}, y_{goal}	-1.5	1.5	m
Initial yaw angle ψ_0	0	2π	rad

Table 2.2: Bounds of the randomized task parameters.

Chapter 3

Reinforcement Learning

This chapter presents a data-driven approach to solving the task, based on reinforcement learning (RL). The RL literature dates back several decades [18], but its major successes came only with the growth of available computational power, and numerous applications have emerged since.

RL is a natural choice for the problem described in the *Task Description* section. One of its key properties is that it does not require an exact mathematical model of the system; a simulator that accurately reproduces the system's behavior is sufficient. This is particularly advantageous here, since the dynamic model of the drone–manipulator system carrying an unknown payload is hard to obtain, whereas an accurate MuJoCo model of it is available. Moreover, since the policy is trained in simulation, techniques such as domain randomization can be used to make it robust to modeling errors, which facilitates the transfer of the learned policy from simulation to the real system [27]. In this thesis, the widely used Proximal Policy Optimization (PPO) algorithm [17] is applied. The first part of the chapter introduces the RL methodology in the context of the thesis.

Despite these advantages, RL-based control also has several shortcomings. First, training can be very slow, depending on the complexity of the task. Second, in more complex environments, the agent may get stuck in local optima that do not produce the desired behavior. To address these issues, the second and third parts of this chapter introduce curriculum learning (CL) [13] and hierarchical reinforcement learning (HRL) [9].

3.1 Reinforcement Learning Formulation

The notation of [18] is adopted throughout this chapter. The interaction between the agent and its environment is modeled as an episodic Markov decision process (MDP)

$$m = (\mathcal{S},\, \mathcal{A},\, \mathcal{P},\, r,\, \rho_0,\, \mathcal{S}_f)\,, \tag{3.1}$$

where

- $\mathcal{S}$ is the state space,
- $\mathcal{A}$ is the action space,
- $\mathcal{P}(s' \mid s, a)$ is the transition function, i.e., the probability of reaching state s' after taking action a in state s,
- $r(s, a, s')$ is the reward function,
- ρ_0 is the initial state distribution, and
- $\mathcal{S}_f$ is the set of terminal states.

At each time step t, the environment is in state $S_t \in \mathcal{S}$. The agent, however, does not have access to the full state: the physical parameters of the payload, for instance, are not observed. Instead,

it receives an observation O_t, which contains only part of the information in S_t. Based on this observation, the agent selects an action $A_t \in \mathcal{A}$ according to its policy

$$A_t \sim \pi_\zeta(\cdot \mid O_t)\,, \tag{3.2}$$

where π_ζ is a stochastic policy represented by a neural network with parameters ζ. In response:

- the environment transitions to a new state $S_{t+1} \sim \mathcal{P}(\cdot \mid S_t, A_t)$,
- it returns a scalar reward $R_{t+1} = r(S_t, A_t, S_{t+1})$, and
- the episode, which starts from an initial state $S_0 \sim \rho_0$, lasts until a terminal state in $\mathcal{S}_f$ is reached or the time limit is exceeded, at time step T.

The goal of the agent is to maximize the return, i.e., the discounted cumulative reward collected from time step t until the end of the episode,

$$G_t = \sum_{t'=t}^{T-1} \gamma^{t'-t} R_{t'+1}, \tag{3.3}$$

where $\gamma \in [0, 1]$ is the discount factor. Accordingly, training seeks the policy parameters that maximize the expected return from the initial state:

$$J(\zeta) = \mathbb{E}_{\pi_\zeta}[G_0]\,, \qquad \zeta^\star = \arg\max_\zeta J(\zeta). \tag{3.4}$$

In this thesis, $J(\zeta)$ is maximized with PPO [17].

All policies in this thesis are trained with the same setup: 8 parallel training environments collect the experience, and a separate evaluation environment is used to monitor the training. In the training figures, the training return is the mean return of the last 100 completed training episodes, while every $4 \cdot 10^4$ environment steps, i.e., 5000 steps per training environment, the deterministic policy is evaluated on 5 episodes in the evaluation environment. The PPO implementation of Stable-Baselines3 [15] is used for all policies with the same hyperparameters, which are given in Appendix A.1.

The following subsections define the observation, the action, the reward, and the episode of the agent that solves the transportation task.

3.1.1 Observation

The observation $O_t \in \mathbb{R}^{35}$ consists of

$$O_t = \left[p^T,\ v^T,\ \eta^T,\ \omega^T,\ (w_i - p)^T,\ \Delta_2^T, \ldots, \Delta_7^T,\ l_1,\ l_2\right]^T, \tag{3.5}$$

where

- $p \in \mathbb{R}^3$ is the position of the drone,
- $v \in \mathbb{R}^3$ is the linear velocity of the drone,
- $\eta = [\phi,\, \theta,\, \psi]^T$ contains the roll, pitch, and yaw angles of the drone,
- $\omega \in \mathbb{R}^3$ is the angular velocity of the drone in the body frame, as in (2.4),
- $w_i - p \in \mathbb{R}^3$ is the position of the active waypoint w_i, defined in the *Reward* subsection, relative to the drone,
- $\Delta_j = p_{\text{payload}} - p_{s_j} \in \mathbb{R}^3$ is the position of the payload relative to the j-th segment of the manipulator, $j = 2, \ldots, 7$, and
- l_1, l_2 are the normalized lengths of the two tendons.

The angular velocity ω is expressed in the body frame, since it is measured in this frame by the gyroscope of the drone, and the motors rotate the drone about its body axes. All other vector quantities are expressed in the world frame, in which the waypoints and the payload are given.

3.1.2 Action

The action $A_t = [a_1, \ldots, a_6]^T \in [-1,\, 1]^6$ consists of

- four normalized motor commands $a_1, \ldots, a_4$, and
- two tendon commands a_5 and a_6.

To simplify the learning of the grasping motion, the tendons are not controlled directly by the agent. Instead, they are actuated as follows:

- only the closing magnitude μ is learned, from a_5:

$$\mu = \frac{a_5 + 1}{2} \in [0,\, 1], \tag{3.6}$$

  where $\mu = 0$ corresponds to the relaxed and $\mu = 1$ to the fully closed hook,
- the curling direction $\sigma \in \{-1,\, 1\}$ determines toward which side of the drone the hook curls; it is not learned, but determined from the geometry, namely from the side of the payload relative to the second segment of the manipulator, expressed in the body frame of the drone, since the hook has to curl toward the payload, and learning this discrete choice would only slow down the training,
- the two tendons are actuated symmetrically with the commands $\sigma\mu$ and $-\sigma\mu$, i.e., one tendon is shortened while the other one is lengthened by the same amount; therefore, a single command is sufficient, and a_6 is not used, and
- the tendons are actuated only after the payload has come within $R_{\text{payload}} + 0.02\,\text{m}$ of the second segment of the manipulator and the grasp waypoint has been activated; otherwise, they are relaxed.

3.1.3 Reward

The reward at each time step consists of a dense shaping term r_{shape}, sparse bonuses r_{bonus} for completing the subtasks, and a termination penalty r_{term}:

$$R_{t+1} = r_{\text{shape}} + r_{\text{bonus}} + r_{\text{term}}. \tag{3.7}$$

Both r_{shape} and r_{bonus} are built around a sequence of waypoints, of which exactly one, w_i, is active at a time. The waypoints follow each other in the order

1. take-off,
2. approach above the payload,
3. grasp pose, computed from the position and the radius of the payload, and
4. goal position.

The active waypoint w_i is considered reached if the horizontal and vertical position errors of the drone satisfy

$$e_{xy} = \|p_{xy} - w_{i,xy}\| < \varepsilon_{\text{wp}}, \qquad e_z = |p_z - w_{i,z}| < \frac{\varepsilon_{\text{wp}}}{5}, \tag{3.8}$$

where $\varepsilon_{\text{wp}} = 0.1\,\text{m}$ is the waypoint tolerance. Reaching a waypoint activates the next one, except for the grasp pose: there, the next waypoint is activated only once the payload is grasped, as defined below.

To measure how tightly the manipulator is wrapped around the payload, the cumulative segment distance is defined as

$$d_{\text{seg}} = \left\| \left[\|\Delta_2\| - R_{\text{payload}} \quad \cdots \quad \|\Delta_7\| - R_{\text{payload}} \right] \right\|, \tag{3.9}$$

i.e., the norm of the gaps between the surface of the cylinder and the manipulator segments. The payload is considered grasped when $d_{\text{seg}} < 0.1\,\text{m}$.

The shaping term, applied at every time step, is

$$r_{\text{shape}} = -e_z - 0.1\, e_{xy} - 0.03\, \left\| [\phi,\, \theta]^T \right\| - 0.06\, \|\omega\| - 0.03\, d_{\text{seg}}, \tag{3.10}$$

where e_{xy} and e_z are the waypoint errors in (3.8), and d_{seg} is defined in (3.9). It drives the drone toward the active waypoint, penalizes tilting and rapid rotations, and encourages the manipulator to stay close to the payload.

The sparse bonuses r_{bonus} depend on the active waypoint and are summarized in Table 3.1. Finally, the termination penalty is $r_{\text{term}} = -100$ if the episode is terminated, as described in the *Episode* subsection, and $r_{\text{term}} = 0$ otherwise.

Active waypoint	Condition	Bonus
Take-off	waypoint reached	+20
Approach	waypoint reached	+20
Grasp pose	waypoint reached	+0.1
	$d_{\text{seg}} < 0.1\,\text{m}$ (payload grasped)	+5
Goal	$d_{\text{seg}} < 0.15\,\text{m}$ (payload carried)	+3
	waypoint reached	+3

Table 3.1: Sparse reward bonuses r_{bonus} depending on the active waypoint.

3.1.4 Episode

At the beginning of each episode, the drone is placed level and at rest at $[0,\, 0,\, 0.5]^T\,\text{m}$, with a random yaw angle, and the payload and the goal are generated randomly. An episode is terminated if

- the drone crashes, i.e., $p_z < 0$,
- the drone flips over, i.e., $|\phi| > \pi/2$ or $|\theta| > \pi/2$, or
- the payload is lost, i.e., its distance from the second segment of the manipulator exceeds 1 m while the goal waypoint is active.

Otherwise, the episode is truncated after 10 s.

3.1.5 Flat Agent

In the rest of this thesis, the agent defined by the observation (3.5), the action of Section 3.1.2, the reward (3.7), and the episode of Section 3.1.4 is referred to as the *flat agent*, since it solves the whole transportation task with a single, non-hierarchical policy π_ζ, directly commanding the motors of the drone.

3.2 Curriculum Learning

The main limitation of the flat agent is its slow training: the agent has to learn the whole task at once, from the take-off to the delivery of the payload, and since the bonuses of the later subtasks are only collected once the earlier ones are mastered, it needs a very large number of environment steps to discover the desired behavior. Curriculum learning (CL) [13] addresses this problem: instead of training the agent directly on the full task, a sequence of tasks of increasing difficulty is defined. Training starts with an easy task that the agent can learn quickly, and the task is made progressively harder, with each stage starting from the policy learned in the previous one. Since the agent always builds on the skills acquired in the earlier stages, it reaches the desired behavior considerably faster than when trained on the full task from scratch.

3.2.1 Formulation

Following [13], each task of the curriculum is an MDP of the form (3.1):

$$m_k = (\mathcal{S},\, \mathcal{A},\, \mathcal{P}_k,\, r_k,\, \rho_{0,k},\, \mathcal{S}_{f,k})\,, \qquad k = 1, \ldots, n. \tag{3.11}$$

The last task m_n is the target task, i.e., the full transportation task of the flat agent. Since the observation and action spaces are identical in all tasks, the same policy network π_ζ can be used throughout the curriculum. The tasks, however, may differ in all other elements of the MDP:

- the transition function $\mathcal{P}_k$, since the ranges from which the mass and the radius of the payload are sampled change the dynamics of the system,
- the reward function r_k, since the terms related to grasping and carrying the payload are only present once grasping is required,
- the initial state distribution $\rho_{0,k}$, through the sampling range of the goal position, and
- the set of terminal states $\mathcal{S}_{f,k}$, since losing the payload terminates the episode only in the target task.

The curriculum applied in this thesis is a task-level sequence curriculum, i.e., an ordered list of tasks

$$C = [\, m_1,\, m_2,\, \ldots,\, m_n \,], \tag{3.12}$$

where the intermediate tasks $m_1, \ldots, m_{n-1}$ are simplified versions of the target task m_n. Training starts with task m_1, and the policy parameters ζ are always carried over when the task changes. The current task is selected adaptively, based on the return G_0 in (3.3), averaged over the last N episodes, denoted by $\bar{G}_0$:

$$m_k \to \begin{cases} m_{k+1}, & \text{if } \bar{G}_0 \geq \delta_{\text{adv}} \text{ and } k < n, \\ m_{k-1}, & \text{if } \bar{G}_0 \leq \delta_{\text{ret}} \text{ and } k > 1, \\ m_k, & \text{otherwise,} \end{cases} \tag{3.13}$$

where δ_{adv} and δ_{ret} are the thresholds for advancing to the next task and retreating to the previous one, respectively. The retreat rule prevents the agent from remaining in a task that turned out to be too difficult, e.g., due to forgetting previously learned skills. Each of the 8 training environments applies (3.13) to its own last N episodes; the curriculum task shown in the training figures is the highest task reached by the training environments, and the evaluation environment always uses this task.

3.2.2 Training with and without CL

To evaluate CL, the training of the flat agent with CL is compared to training directly on the target task. The curriculum of the flat agent consists of $n = 4$ tasks, summarized in Table 3.2. The parameters of the task switching rule (3.13) are given in Table 3.3.

Task	Description	Grasping	m_{payload} [kg]	R_{payload} [m]	$x_{\text{goal}}, y_{\text{goal}}$ [m]
m_1	Waypoint following	no	–	–	$[-1.5, 1.5]$
m_2	Grasping a nearly fixed payload	yes	0.05	[0.02, 0.025]	$[-1, 1]$
m_3	Grasping a randomized payload	yes	[0.01, 0.25]	[0.02, 0.04]	$[-1, 1]$
m_4	Full task	yes	[0.01, 0.25]	[0.02, 0.04]	$[-1.5, 1.5]$

Table 3.2: Tasks of the curriculum of the flat agent.

In tasks m_1–m_3, the approach waypoint above the payload is omitted, and losing the payload does not terminate the episode. Both are introduced only in the target task m_4.

Parameter	Symbol	Value
Number of tasks	n	4
Averaging window	N	100 episodes
Advancing threshold	δ_{adv}	1000
Retreating threshold	δ_{ret}	-1000

Table 3.3: Parameters of the task switching rule.

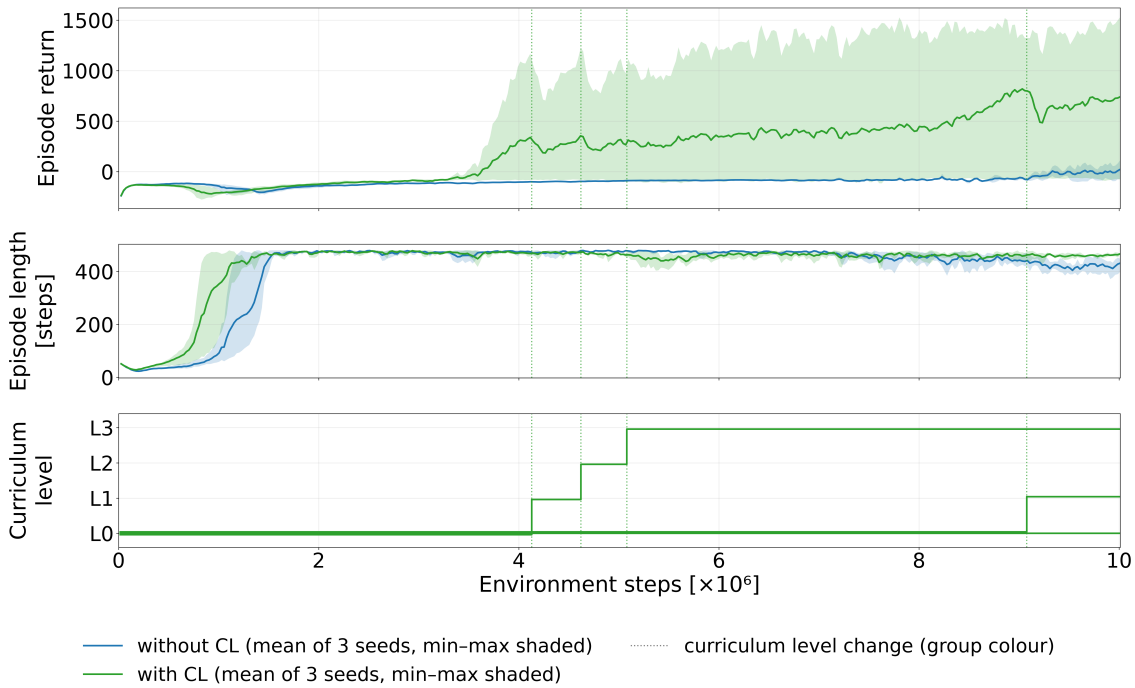


Figure 3.1: Training of the flat agent without CL (blue) and with CL (green): training return (top), episode length (middle), and curriculum task (bottom). Solid lines: mean of three random seeds; shaded areas: min–max range over the seeds; dotted vertical lines: task switches. The levels L0–L3 correspond to the tasks m_1–m_4.

As can be seen in Figure 3.1, the flat agent trained with CL reaches a high episodic return considerably faster than the agent trained without it. However, the learning process shows a high variance across the random seeds: while on some seeds the agent converges quickly, on others the training does not converge within the given number of time steps. Thus, the benefit of CL, although clearly visible on average, is not guaranteed for every individual training run. During training, the agent

only ever advanced to the next task, i.e., the retreat rule in (3.13) was never triggered. Since CL improved the learning speed in this example, all policies in the remainder of this thesis are trained with CL.

3.3 Hierarchical Reinforcement Learning

Although CL speeds up learning, the fact that the flat agent commands the motors of the drone directly leads to two problems. First, the agent has to learn not only how to solve the task but also how to stabilize the drone, which slows down learning. Second, the learned policy may not transfer well to the real system, since the real rotors may not follow commands with such a high bandwidth, and unmodeled phenomena, e.g., motor dynamics and control delays, may cause the motors to track the commanded revolutions per minute (RPMs) inaccurately.

To address these issues, a hierarchical reinforcement learning (HRL) approach [9] is proposed, in which two agents work together: the low-level agent tracks a reference velocity and yaw angle, while the high-level agent plans the motion by providing these references.

3.3.1 Formulation

Following [9], the applied hierarchy is a feudal one: a high-level policy sets subgoals for a low-level policy, which in turn determines how these subgoals are achieved. In this thesis, the subgoal is a reference velocity and a reference yaw angle of the drone. Both levels are formulated as MDPs of the form (3.1), and the hierarchy consists of

- a high-level policy π_ζ^{H} with parameters ζ, which, similarly to the flat agent, solves the transportation task: based on the observation O_t^{H}, it selects the references and the tendon commands,
$$A_t^{\mathrm{H}} \sim \pi_\zeta^{\mathrm{H}}\big(\cdot \mid O_t^{\mathrm{H}}\big)\,, \tag{3.14}$$
- a low-level policy π_ν^{L} with parameters ν, which computes the motor commands based on its own observation O_t^{L}, containing the references given by the high level:
$$A_t^{\mathrm{L}} \sim \pi_\nu^{\mathrm{L}}\big(\cdot \mid O_t^{\mathrm{L}}\big)\,. \tag{3.15}$$

In the following, the quantities of the high-level and the low-level policy are marked with the superscripts H and L, respectively. The interaction of the two policies with each other and with the environment is shown in Figure 3.2.

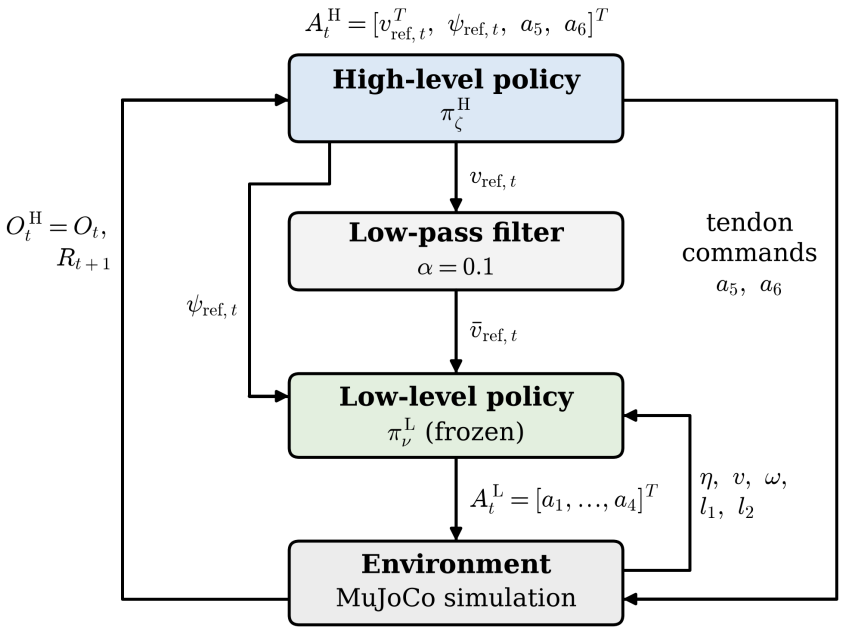


Figure 3.2: Structure of the hierarchical agent. The high-level policy π_ζ^{H} receives the same observation and reward as the flat agent, and provides the reference velocity and yaw angle to the frozen low-level policy π_ν^{L}, as well as the tendon commands to the environment. The reference velocity is smoothed by the low-pass filter (3.19). The low-level policy computes the motor commands based on the state of the drone and the references.

The two policies are trained separately. First, the low-level policy is trained on a velocity tracking task, and then its parameters ν are frozen. Afterwards, the high-level policy is trained on the transportation task, while the frozen low-level policy executes its references. Since the low-level policy handles the stabilization of the drone, the high-level policy only has to learn *where* the drone should move and *when* the manipulator should close. Moreover, as the interface between the two levels is a velocity and yaw reference, the high-level policy can be replaced by any other planner that provides such references, e.g., the model predictive controller presented in the *Model Predictive Control* chapter.

The following subsections define the low-level policy in the same structure as the flat agent.

3.3.2 Low-Level Observation

The observation $O_t^{\mathrm{L}} \in \mathbb{R}^{15}$ of the low-level policy consists of

$$O_t^{\mathrm{L}} = \left[v^T,\ \eta^T,\ \omega^T,\ v_{\mathrm{ref}}^T,\ \psi_{\mathrm{ref}},\ l_1,\ l_2\right]^T, \tag{3.16}$$

where

- $v \in \mathbb{R}^3$ is the linear velocity of the drone,
- $\eta = [\phi,\ \theta,\ \psi]^T$ contains the roll, pitch, and yaw angles of the drone,
- $\omega \in \mathbb{R}^3$ is the angular velocity of the drone in the body frame,
- $v_{\mathrm{ref}} \in \mathbb{R}^3$ is the reference velocity,
- ψ_{ref} is the reference yaw angle, and

- l_1, l_2 are the normalized lengths of the two tendons.

3.3.3 Low-Level Action

The action $A_t^{\mathrm{L}} = [a_1, \ldots, a_4]^T \in [-1,\, 1]^4$ consists of the four normalized motor commands, as in the action of the flat agent. The tendons are not controlled by the low-level policy; instead, they are set by the environment:

- if a payload is carried, the hook is fully closed toward the side of the payload, and
- otherwise, the tendons are relaxed.

3.3.4 Low-Level Reward

Similarly to (3.7), the reward of the low-level policy consists of a dense shaping term, a sparse bonus, and a termination penalty:

$$R_{t+1}^{\mathrm{L}} = r_{\mathrm{shape}}^{\mathrm{L}} + r_{\mathrm{bonus}}^{\mathrm{L}} + r_{\mathrm{term}}^{\mathrm{L}}, \qquad r_{\mathrm{shape}}^{\mathrm{L}} = -e_v - e_\psi - 0.1\,(|\phi| + |\theta|)\,, \tag{3.17}$$

where $e_v = \|v - v_{\mathrm{ref}}\|$ is the velocity tracking error and $e_\psi = |\psi - \psi_{\mathrm{ref}}|$ is the yaw error. The shaping term drives the drone toward the reference velocity and yaw angle, and penalizes tilting. The bonus $r_{\mathrm{bonus}}^{\mathrm{L}} = 0.5$ rewards accurate tracking, $e_v < 0.025\,\mathrm{m/s}$, and it is zero otherwise. The termination penalty is $r_{\mathrm{term}}^{\mathrm{L}} = -100$ if the episode is terminated, and zero otherwise.

3.3.5 Low-Level Episode

At the beginning of each episode, the drone is placed level and at rest at $[0,\, 0,\, 12.8]^T\,\mathrm{m}$, i.e., high enough above the ground to avoid collisions while tracking the reference, with a random yaw angle in $[-\pi,\, \pi)$. The reference yaw angle is set to $\psi_{\mathrm{ref}} = 0$, so that the policy also learns to turn toward the reference. The reference velocity is sampled from a mixture of uniform distributions, so that both hovering and fast motions are sufficiently represented:

$$v_{\mathrm{ref}} \sim \begin{cases} \mathcal{U}\big([-0.1,\, 0.1]^3\big)\ \mathrm{m/s}, & \text{with probability } 0.4, \\ \mathcal{U}\big([-0.5,\, 0.5]^3\big)\ \mathrm{m/s}, & \text{with probability } 0.3, \\ \mathcal{U}\big([-1,\, 1]^3\big)\ \mathrm{m/s}, & \text{with probability } 0.3. \end{cases} \tag{3.18}$$

To make the policy robust to the payload, in about half of the episodes a payload is attached to the drone, already grasped in the closed hook, with a random mass and radius, whose ranges are given by the curriculum in Table 3.4. An episode is terminated if the drone flips over, i.e., $|\phi| > \pi/2$ or $|\theta| > \pi/2$, and truncated after 10 s otherwise.

3.3.6 Low-Level Agent Results

As stated in the *Curriculum Learning* section, the low-level policy is also trained with a curriculum, following the formulation given in (3.11)–(3.13). Its tasks, denoted by m_k^{L}, are summarized in Table 3.4, and the parameters of the task switching rule are given in Table 3.5.

Task	Description	Random yaw	Payload	m_{payload} [kg]	R_{payload} [m]
m_1^{L}	Velocity tracking without payload	no	no	–	–
m_2^{L}	Velocity tracking with a nearly fixed payload	yes	yes	0.05	[0.02, 0.03]
m_3^{L}	Full task	yes	yes	[0.01, 0.25]	[0.02, 0.04]

Table 3.4: Tasks of the curriculum of the low-level policy.

Parameter	Symbol	Value
Number of tasks	n	3
Averaging window	N	20 episodes
Advancing threshold	δ_{adv}	110
Retreating threshold	δ_{ret}	-1000

Table 3.5: Parameters of the task switching rule of the low-level policy.

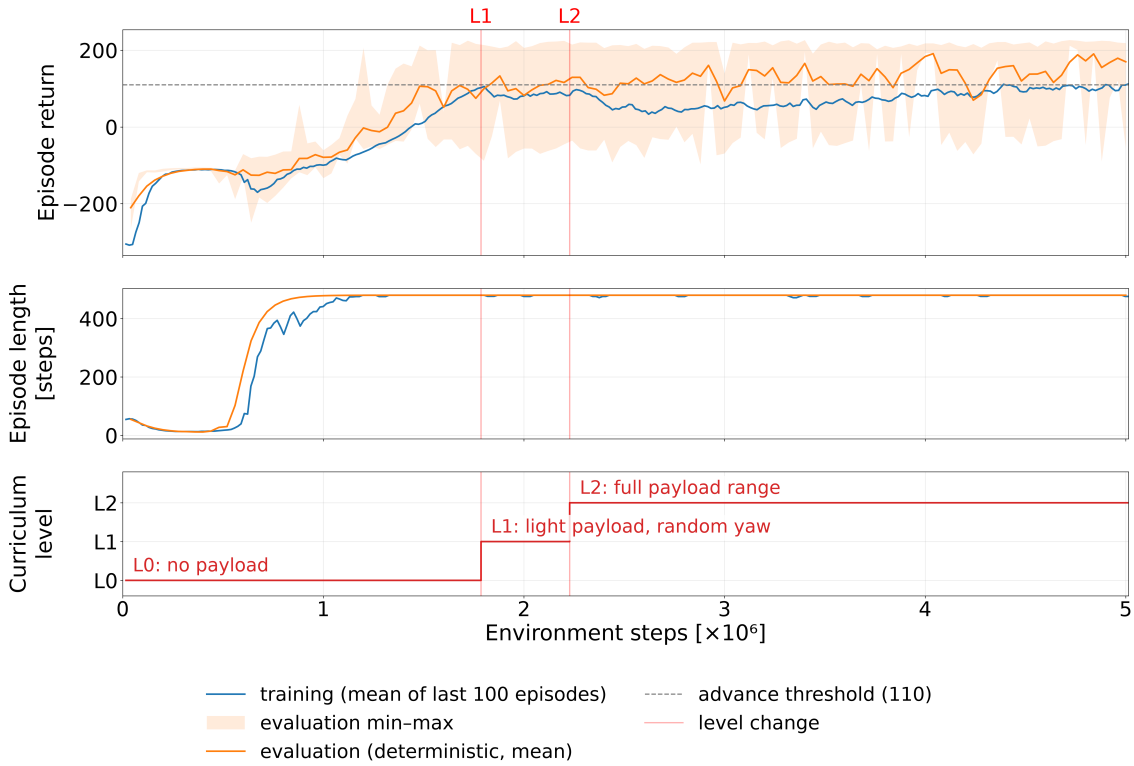


Figure 3.3: Training of the low-level policy: return (top), episode length (middle), and curriculum task (bottom). Blue: training return; orange and shaded area: mean and min–max range of the evaluation episodes; light red vertical lines: task switches.

As can be seen in Figure 3.3, the training of the low-level policy converges: by the end of the training, the mean training return stabilizes at around 100, while the evaluation return reaches about 150–180. The episode length quickly reaches its maximum of 480 steps, i.e., 10 s at 48 Hz, which means that the drone no longer flips over. The effect of the curriculum is also visible: after the switch to the full task m_3^{L}, the return drops, since the policy has to learn velocity tracking under more difficult conditions, with payloads of widely varying mass and radius. By the end of the training, however, the policy successfully adapts to these conditions.

To evaluate the trained low-level policy, 200 test runs were performed with a piecewise constant reference velocity profile of 12 s, in which the reference changes at $t = 1$, 3, 5, 6.5, 8.5, and 10 s. In each run, the initial yaw angle of the drone is random, and in 93 runs a payload with random mass and radius is attached to the drone, while the remaining 107 runs are performed without payload. Unlike in training, the reference changes during the episode, and the episode is longer than 10 s. The results are shown in Figures 3.4–3.6.

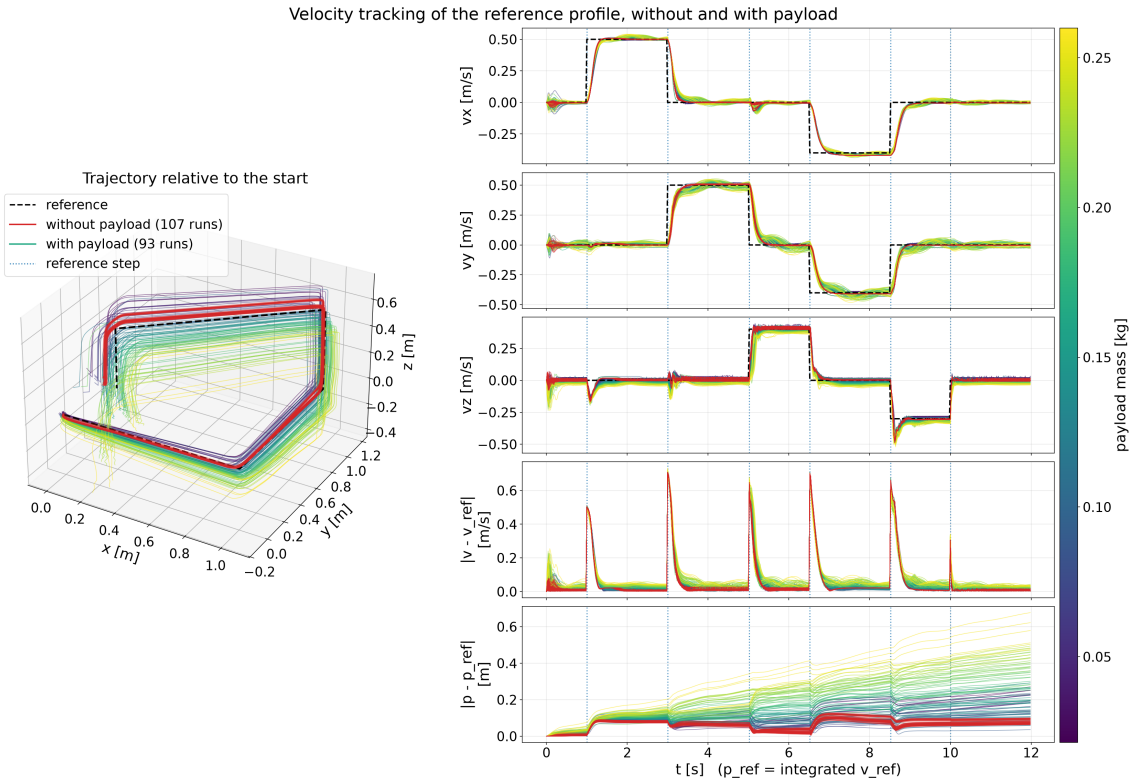


Figure 3.4: Velocity tracking of the low-level policy over 200 test runs. Left: trajectory relative to the starting position. Right, from top to bottom: velocity components v_x, v_y, v_z, velocity error norm $\|v - v_{\text{ref}}\|$, and position error $\|p - p_{\text{ref}}\|$ with respect to the integrated reference velocity. Red: runs without payload; other colors: runs with payload, colored by the payload mass. Black dashed line: reference; dotted vertical lines: reference steps.

As can be seen in Figure 3.4, the low-level policy tracks the reference velocity well in all runs. After each reference step, the velocity settles within about 0.3–0.5 s, and large velocity errors appear only at the steps themselves. A horizontal step also causes a short vertical transient, since tilting the drone temporarily reduces the vertical component of the thrust. Although the policy was trained only with constant references, it also tracks a changing reference reliably, which is essential for its use under the high-level policy.

The effect of the payload is most visible in the position error: without payload, it remains around 0.1 m, while for the heaviest payloads it grows up to about 0.5–0.7 m by the end of the run. This drift does not affect the overall task, since the position is controlled by the high level.

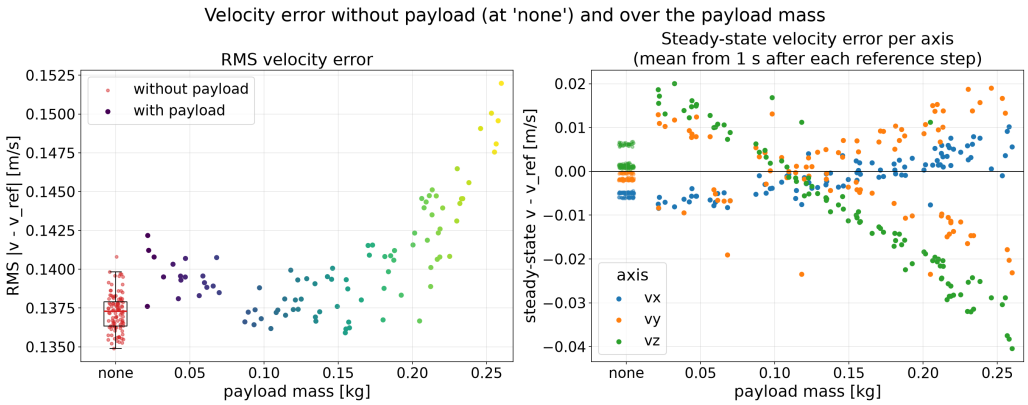


Figure 3.5: Velocity tracking error of the low-level policy as a function of the payload mass; "none" denotes the runs without payload. Left: RMS velocity error over the whole run. Right: steady-state velocity error per axis, averaged from 1 s after each reference step.

Figure 3.5 quantifies the effect of the payload mass. The RMS velocity error, which is dominated by the transients after the reference steps, increases only moderately with the payload mass, from about 0.14 m/s without payload to about 0.152 m/s for the heaviest payloads. The steady-state error explains the position drift: without payload, it stays below 0.01 m/s on every axis, whereas for the heaviest payloads, the vertical error reaches about −0.04 m/s, i.e., the policy does not fully compensate for the additional weight. Nevertheless, this error remains small compared to the reference velocities of 0.3–0.5 m/s.

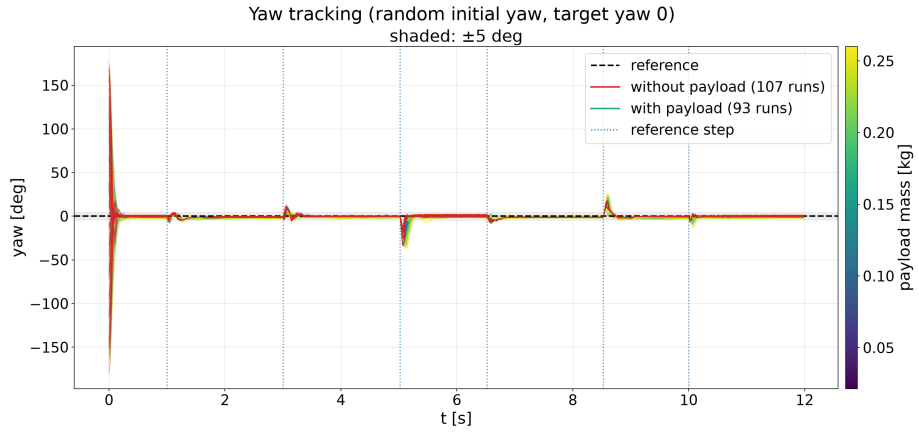


Figure 3.6: Yaw tracking of the low-level policy over 200 test runs, with a random initial yaw angle and a reference yaw angle of $\psi_{\text{ref}} = 0$. Red: runs without payload; other colors: runs with payload, colored by the payload mass. The gray band indicates ±5°; dotted vertical lines mark the steps of the reference velocity.

Finally, Figure 3.6 shows the yaw tracking. Starting from a random initial yaw angle in the range of ±180°, the drone turns to the reference yaw angle within about 0.3 s in all runs, and then keeps the yaw error within ±5°. The steps of the reference velocity cause short yaw disturbances of up to about 35°, mainly at the vertical steps, where the collective thrust changes abruptly; they are somewhat larger with heavier payloads, but decay within about 0.3 s in every run. Overall, the evaluation shows that the low-level policy provides fast and robust velocity and yaw tracking, both without and with payloads of various masses.

3.3.7 High-Level Policy

The high-level policy solves the same transportation task as the flat agent: its observation $O_t^{\mathrm{H}} = O_t$ is given in (3.5), and its reward, initial state distribution, and termination conditions are also the same. It differs from the flat agent only in the action A_t^{H}, which consists of

- the reference velocity $v_{\mathrm{ref},t} \in \mathbb{R}^3$ and the reference yaw angle $\psi_{\mathrm{ref},t}$ of the low-level policy (3.16), and
- the two tendon commands a_5 and a_6, which are processed in the same way as in the action of the flat agent, see (3.6).

The reference velocity is not passed to the low-level policy directly, but it is smoothed by a first-order low-pass filter:

$$\bar{v}_{\mathrm{ref},t+1} = \bar{v}_{\mathrm{ref},t} + \alpha \left(v_{\mathrm{ref},t} - \bar{v}_{\mathrm{ref},t} \right), \qquad \alpha = 0.1, \tag{3.19}$$

and the low-level policy receives the filtered reference $\bar{v}_{\mathrm{ref},t}$ in place of v_{ref} in (3.16). The filter regularizes the action of the high-level policy: abrupt changes of the commanded velocity are smoothed out, so the low-level policy receives a smooth reference, which reduces the oscillations of the drone and of the swinging payload.

3.3.8 High-Level Agent Results

The high-level policy is trained with the same curriculum as the flat agent, given in Tables 3.2 and 3.3, so its training can be directly compared to that of the flat agent. Figure 3.7 shows the training of the high-level policy, and Figure 3.8 compares it to the flat agent, both trained on three random seeds.

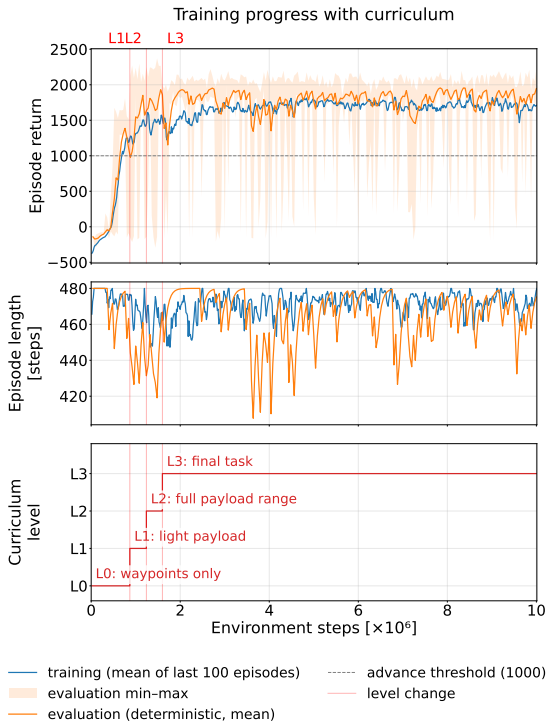


Figure 3.7: Training of the high-level policy: return (top), episode length (middle), and curriculum task (bottom). Blue: training return; orange and shaded area: mean and min–max range of the evaluation episodes; gray dashed line: advancing threshold δ_{adv}; light red vertical lines: task switches.

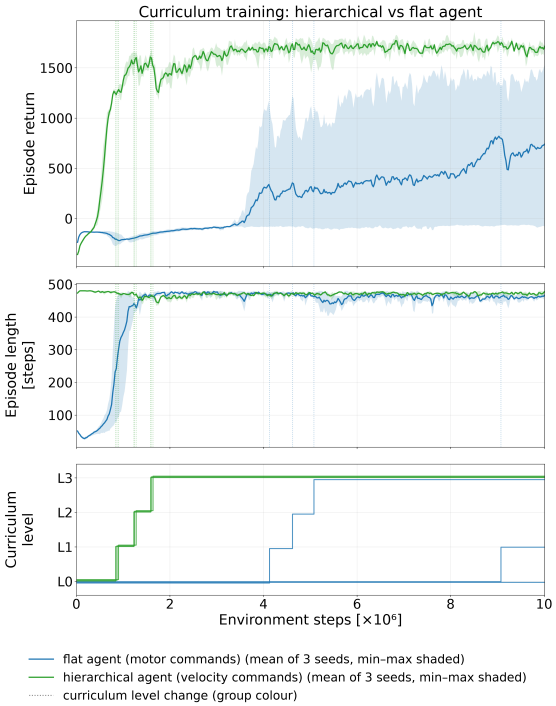


Figure 3.8: Training of the flat agent (blue) and the hierarchical agent (green) with CL: training return (top), episode length (middle), and curriculum task (bottom). Solid lines: mean of three random seeds; shaded areas: min–max range over the seeds; dotted vertical lines: task switches of the individual seeds. In both figures, the levels L0–L3 correspond to the tasks m_1–m_4 in Table 3.2.

As can be seen in Figure 3.7, the training of the high-level policy converges quickly: the agent reaches the target task m_4 after about $1.6 \cdot 10^6$ steps, and after about $3 \cdot 10^6$ steps, the return stabilizes at around 1700 in training and 1700–1900 in evaluation. The episode length stays close to its maximum during the whole training, since the low-level policy keeps the drone stable from the first episode. However, occasional low-return, shorter evaluation episodes remain even at the end of the training, which motivates the detailed validation of the policy.

As can be seen in Figure 3.8, the hierarchical agent learns considerably faster and more reliably than the flat agent. All three of its seeds reach the target task m_4 within about $1.6 \cdot 10^6$ steps, and the return converges to around 1700 with a narrow spread. In contrast, the flat agent stays in the first task for at least $4 \cdot 10^6$ steps, and by the end of the training, only one of its seeds reaches the target task, while one never leaves the first task, resulting in a mean return of only about 700.

The main reason is that the flat agent first has to learn to stabilize the drone, as shown by its short episodes during the first $1.3 \cdot 10^6$ steps, whereas the hierarchical agent relies on the already trained low-level policy and can spend the whole training on the transport task. Note that this comparison does not include the training steps of the low-level policy.

The detailed validation of the high-level policy is presented in the *Comparison* chapter, where it is compared to the model predictive controller and to the RL agent with the predictive safety filter.

Chapter 4

Model Predictive Control

This chapter presents model predictive control (MPC), another method capable of solving the payload transportation task. MPC is widely applied to drone–manipulator systems, for example, for payload transportation with a hook [3], which makes it a natural choice for this work as well.

MPC has several advantages. First, constraints on the states and the control inputs, such as velocity limits, can be handled explicitly in the optimization problem, which leads to safe and predictable behavior. Second, since MPC relies on a model of the system rather than on previously collected experience, it can also perform well in situations that were not encountered during its design [16]. Third, the smoothness of the control inputs can be enforced directly, e.g., by penalizing their magnitude and their changes in the cost function.

Nevertheless, MPC also has drawbacks. Most importantly, it requires an accurate model of the system, and its performance is sensitive to modeling errors. Furthermore, to solve the optimization problem at every control step, the model has to be rolled out over the prediction horizon, and its Jacobian matrices have to be computed, which is computationally demanding, especially on drones with limited onboard resources. The computational demand can be reduced by shortening the prediction horizon, but this may come at the expense of control performance.

Since a first-principles model of the drone–manipulator system is hard to obtain, a data-driven model is applied: a feedforward residual neural network (ResNet) [5] describes the closed-loop behavior of the drone controlled by the low-level policy, i.e., how the drone responds to a given reference velocity. Thus, the MPC replaces the high-level policy of the hierarchical architecture. The first part of this chapter introduces the formulation of the MPC, while the second part presents the identification procedure.

4.1 Model Predictive Control Formulation

MPC is based on numerical optimization: at each control step, the future behavior of the system is predicted over a finite horizon with a model, a cost function is minimized over the future control inputs subject to constraints, and only the first element of the resulting input sequence is applied to the system. At the next control step, the procedure is repeated from the newly measured state, i.e., the horizon is shifted by one step.

The MPC receives the same information about the task as the high-level policy of Section 3.3.7. It runs at the control frequency of 48 Hz, i.e., with the sampling time $\Delta t = 1/48\,\mathrm{s}$, while the reference yaw angle is kept at $\psi_{\mathrm{ref}} = 0$. The MPC does not follow a precomputed reference trajectory: its target is simply the active waypoint of the environment (take-off, pre-grasp, grasp, and goal waypoint), which is advanced by the environment in the same way as for the RL agents. The grasping is also handled by the environment: the MPC always requests the fully closed hook, and the tendons are actuated according to the same rule as for the RL agents, see Section 3.1.2, i.e., only once the payload is close to the hook and the grasp waypoint is active.

4.1.1 State and Input

The input of the MPC is the normalized velocity command $u_k \in [-1, 1]^3$, i.e., the reference velocity v_{ref} of the low-level policy in (3.19). The index k denotes the control steps, which correspond to the time steps t of the RL formulation. As for the high-level policy, the command is smoothed by the low-pass filter (3.19), so the low-level policy tracks the filtered reference $\bar{v}_{\text{ref},k}$. Since the filter is known exactly, $\bar{v}_{\text{ref},k}$ is included in the state of the model, and its dynamics do not have to be identified.

The state of the model is

$$x = \left[p^T,\ v^T,\ \tilde{\eta}^T,\ \dot{\tilde{\eta}}^T,\ \bar{v}_{\text{ref}}^T,\ \beta^T,\ \dot{\beta}^T\right]^T \in \mathbb{R}^{19}, \tag{4.1}$$

where

- $p \in \mathbb{R}^3$ and $v \in \mathbb{R}^3$ are the position and the velocity of the drone in the world frame,
- $\tilde{\eta} = [\tilde{\phi},\ \tilde{\theta},\ \psi]^T$ is the attitude of the drone, where the tilt angles $\tilde{\phi}$ and $\tilde{\theta}$ are measured about the world x and y axes, computed from the direction of the body z axis, and ψ is the yaw angle; at $\psi = 0$, $\tilde{\phi}$ and $\tilde{\theta}$ coincide with the roll and pitch angles ϕ and θ in η,
- $\dot{\tilde{\eta}}$ is the time derivative of $\tilde{\eta}$, which, unlike the body-frame angular velocity ω in (2.4), directly describes the change of the attitude angles predicted by the model,
- $\bar{v}_{\text{ref}} \in \mathbb{R}^3$ is the filtered reference velocity (3.19), and
- $\beta = [\beta_x,\ \beta_y]^T$ are the swing angles of the payload, i.e., the angles of the line from the hook to the payload in the world x–z and y–z planes, with $\beta = 0$ when the payload hangs straight down, and $\dot{\beta}$ are their rates.

The tilt angles are defined in the world frame, rather than as the roll and pitch angles of the body, so that they are independent of the heading of the drone and consistent with the world-frame velocity: a motion along the world x axis always corresponds to $\tilde{\theta}$, and a motion along the y axis to $\tilde{\phi}$. The state (4.1) contains every quantity on which the closed loop of the low-level policy depends, since the low-level policy observes the attitude, the velocity, the angular velocity, and the filtered reference $\bar{v}_{\text{ref}}$.

4.1.2 Prediction Model

The dynamics of the closed loop change considerably when the payload is lifted. Therefore, two operating modes $\kappa \in \{0, 1\}$ are distinguished: in mode $\kappa = 0$, the drone flies without payload, and the swing states are zero, $\beta = \dot{\beta} = 0$, while in mode $\kappa = 1$, the payload is attached to the hook. A separate model is identified for each mode, and the MPC switches between them when the payload is detected to be attached.

In each mode, the dynamics are described by a model F_κ based on a ResNet, which predicts the change of the state over one control step:

$$x_{k+1} = x_k + F_\kappa(x_k,\ u_k)\,. \tag{4.2}$$

The prediction over the horizon is obtained by applying (4.2) recursively, starting from the measured state, i.e., the states are propagated with forward Euler steps based on the ResNet. The structure and the training of F_κ are presented in Section 4.2.

4.1.3 Optimal Control Problem

The target of the MPC is the active waypoint $w_i \in \mathbb{R}^3$, which is kept constant over the horizon. At the grasp waypoint, the environment actuates the tendons only if the payload is closer to the hook than at the waypoint itself. Therefore, once the drone is within 3 cm of the grasp waypoint,

the target is shifted by the measured offset between the hook and the payload, so that the hook ends up 1 cm from the surface of the payload.

With the horizon of K steps, the MPC solves at every control step the optimal control problem

$$\begin{aligned}
\min_{u_0,\dots,u_{K-1}} \quad & \mathcal{J} = \frac{1}{2}\sum_{k=1}^{K-1} \|p_k - w_i\|_{Q_p}^2 + \frac{\lambda_K}{2}\|p_K - w_i\|_{Q_p}^2 \\
& \quad + \frac{\lambda_u}{2}\sum_{k=0}^{K-1}\|u_k\|^2 + \frac{\lambda_{\Delta u}}{2}\sum_{k=0}^{K-1}\|u_k - u_{k-1}\|^2 && \text{(4.3a)} \\
\text{s.t.} \quad & x_0 = \bar{x}, && \text{(4.3b)} \\
& x_{k+1} = x_k + F_\kappa(x_k, u_k), \qquad k = 0,\dots,K-1, && \text{(4.3c)} \\
& \|u_k\|_\infty \le u_{\max}, \qquad k = 0,\dots,K-1, && \text{(4.3d)} \\
& \left|\tilde{\phi}_k\right| \le \phi_{\max}, \quad \left|\tilde{\theta}_k\right| \le \phi_{\max}, \qquad k = 1,\dots,K, && \text{(4.3e)}
\end{aligned}$$

where

- (4.3a) is the cost function, in which $\|z\|_Q^2 = z^T Q z$:
  - the first term penalizes the distance of the predicted position p_k from the waypoint with the diagonal weight $Q_p = \mathrm{diag}(q_x, q_y, q_z)$, in which the horizontal error is weighted much more than the vertical one, so that the drone stays above the grasp pose while it descends onto the payload,
  - the second term penalizes the position error at the end of the horizon more strongly, with $\lambda_K > 1$, and
  - the third and fourth terms penalize the magnitude of the commands and their change, with u_{-1} being the command applied in the previous control step; they regularize the problem and make the commands smooth, since with less smooth commands, the closed loop of the low-level policy and the swinging payload tends to oscillate,
- (4.3b) is the initial condition, where $\bar{x}$ is the measured state of the system,
- (4.3c) is the prediction model (4.2) of the current mode κ,
- (4.3d) bounds the velocity commands, and
- (4.3e) limits the predicted tilt of the drone, which keeps its motion moderate, reduces the swinging of the payload, and keeps the drone in the region where the model is accurate.

Only the position appears in the cost; the other states, such as the velocity, the attitude, and the swing of the payload, are needed only for an accurate prediction of the position. The parameters of the MPC, given in Table 4.1, were tuned on 40 randomly initialized episodes of the transportation task.

Parameter	Symbol	Value
Sampling time	Δt	1/48 s
Horizon	K	20 steps (0.42 s)
Position weight	(q_x, q_y, q_z)	(100, 100, 10)
Command weight	λ_u	0.5
Command change weight	$\lambda_{\Delta u}$	30
Terminal weight factor	λ_K	5
Command bound	$u_{\max}$	0.8
Tilt limit	$\phi_{\max}$	15°

Table 4.1: Parameters of the MPC.

Since the prediction model is a neural network, (4.3) is a nonconvex optimization problem, for which only a locally optimal solution can be expected [6]. It is solved at every control step with a

Gauss–Newton sequential quadratic programming (SQP) method [8], similar to the SQP solver of the acados software package [22], warm-started from the shifted solution of the previous step.

4.2 System Identification

As mentioned above, the closed loop of the drone and the low-level policy is identified with a feedforward residual neural network (ResNet), an architecture that has proven to be well suited for system identification [5]. This section presents the structure of the identified model F_κ in (4.2), its training, and its validation.

4.2.1 Model Structure

Only the part of the dynamics that cannot be written out is learned. Since the dynamics do not depend on where the drone is, the input of the network is the state (4.1) without the position, $\tilde{x}_k \in \mathbb{R}^{16}$, together with the command u_k of Section 4.1.1:

$$z_k = \left[\tilde{x}_k^T,\ u_k^T\right]^T \in \mathbb{R}^{19}. \tag{4.4}$$

The network f_κ of mode κ follows the ResNet architecture of [5], and its connections are shown in Figure 4.1.

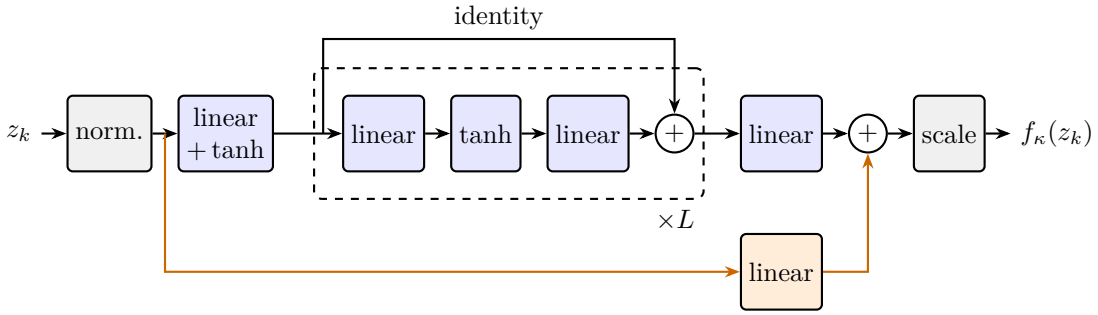


Figure 4.1: Structure of the ResNet f_κ, following [5]. The dashed residual block, whose output is the sum of its input and a correction, is repeated L times, and the linear shortcut (orange) connects the normalized input directly to the output.

Through the identity skip connections, each residual block only adds a correction to its input, which makes the network easy to train, while the linear shortcut represents the linear part of the dynamics directly. The input and the output are normalized with the statistics of the training data, so that all signals have a similar magnitude.

The network predicts the one-step increments of the velocity, the attitude, the attitude rates, and the swing rate:

$$\left[\Delta v_k^T,\ \Delta\tilde{\eta}_k^T,\ \Delta\dot{\tilde{\eta}}_k^T,\ \Delta\dot{\beta}_k^T\right]^T = f_\kappa(z_k) \in \mathbb{R}^{11}. \tag{4.5}$$

From these increments, the state change F_κ in (4.2) is

$$F_\kappa(x_k,\, u_k) = \begin{bmatrix} \Delta t\left(v_k + \frac{1}{2}\Delta v_k\right) \\ \Delta v_k \\ \Delta\tilde{\eta}_k \\ \Delta\dot{\tilde{\eta}}_k \\ \alpha\left(u_k - \bar{v}_{\mathrm{ref},k}\right) \\ \Delta t\left(\dot{\beta}_k + \Delta\dot{\beta}_k\right) \\ \Delta\dot{\beta}_k \end{bmatrix}, \tag{4.6}$$

where the last two rows are zero in mode $\kappa = 0$. That is, the position is integrated with the trapezoidal rule, the filter state follows the known filter (3.19), and the swing angle is integrated from the predicted swing rate. The attitude, on the other hand, is not integrated from its rate, but its increment is predicted directly: the low-level policy makes the roll and yaw angles chatter within a control step, so the sampled attitude rates do not describe the change of the attitude between two samples.

4.2.2 Training

The training data are collected in the transportation task with two kinds of flights:

- flights in which a position P-controller flies the waypoints of the task, while random, piecewise-constant velocity commands are added to its output, except while approaching and grasping the payload; 8 such flights are used for training, half of them with weak and half with strong excitation, and 1 for validation, and
- flights of the trained high-level policy of Section 3.3.7, which provide aggressive commands in the region where the MPC operates; 24 such flights of 15 s are used for training and 4 for validation, with the payload parameters of the target task.

The two kinds of flights complement each other. The flights of the high-level policy contain the fast maneuvers and large tilts that occur when the RL agent flies the task, but they explore the commands only along its own, nearly deterministic trajectories. The randomly excited P-controller flights, on the other hand, cover a wide range of commands and states evenly, including the slow and moderate motions that the MPC itself produces. With only the high-level policy's flights, the model would be accurate only near the trajectories of the RL agent. The time steps in which the hook touches the payload before it is lifted are left out, since the contact forces are not part of the model, and so are the steps of the high-level policy's flights in which the drone is tilted by more than 60°.

A separate network is trained for each mode. Since the MPC relies on multi-step predictions, the network is trained with a multi-step rollout loss: from the measured state x_k, the model (4.2) is applied recursively with the recorded commands $u_k, \dots, u_{k+H-1}$, and the loss is the mean squared error between the predicted and the measured states $x_{k+1}, \dots, x_{k+H}$. The compared states are the velocity, the attitude, and the attitude rates, and in mode $\kappa = 1$ also the swing angles and their rates, each divided by its standard deviation in the training data, so that all of them contribute similarly. Only windows that lie entirely within one mode and contain no left-out steps are used. The rollout horizon H grows linearly from 2 to 24 steps (0.5 s) during the first 40 % of the training, so the network first learns accurate one-step predictions and then stable multi-step ones. The weights with the lowest validation loss at the full horizon are kept, and the training parameters are given in Appendix A.1.

The course of the training is shown in Figure 4.2. While the rollout horizon grows, the loss at the full horizon decreases quickly for both modes, since the network learns to predict over longer and longer windows. Once the full horizon is reached at epoch 60, the validation loss levels off, while the training loss keeps decreasing slowly, i.e., the network starts to fit the training flights more closely than it generalizes. Therefore, the weights with the lowest validation loss are kept, which are found shortly after the full horizon is reached.

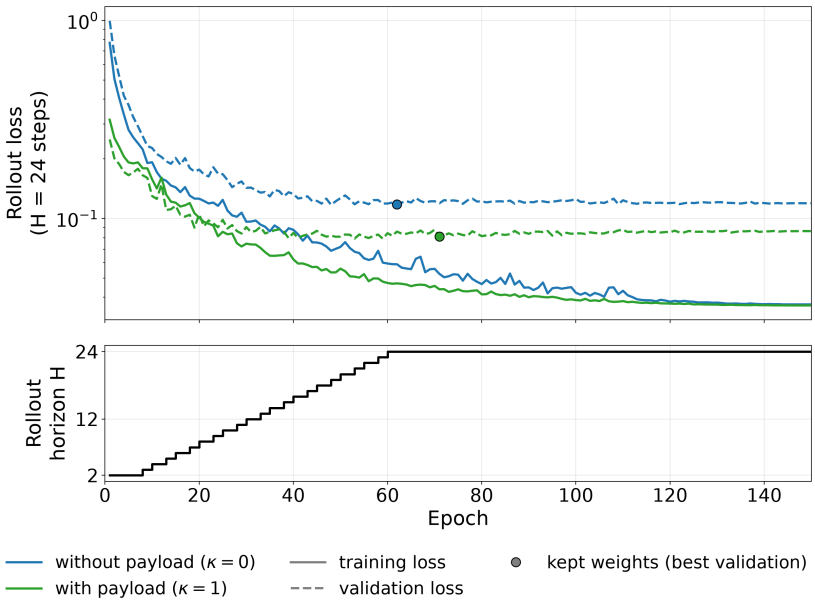


Figure 4.2: Training of the ResNet models. Top: rollout loss over the full horizon of 24 steps on the training and validation data; the dots mark the kept weights. Bottom: rollout horizon H used for training.

4.2.3 Validation of the Identification

The identified model is tested on dedicated test flights, which were not used for training or validation. Each test flight consists of 20 s of free flight without payload, in which random, piecewise-constant velocity commands, held for 0.25–1.5 s, are applied around a fixed point, the pick-up of the payload, and 25 s of similarly excited flight with the payload. The payload mass is set to 13 values between 0.01 and 0.25 kg, each flown three times. Neither the MPC nor the high-level policy is involved: from each measured state, the model is rolled out open loop with the applied commands, and its prediction 0.25, 0.5, and 1 s ahead is compared with the simulation. Note that the horizon of the MPC is 0.42 s, so the 1 s prediction is more demanding than what the MPC requires.

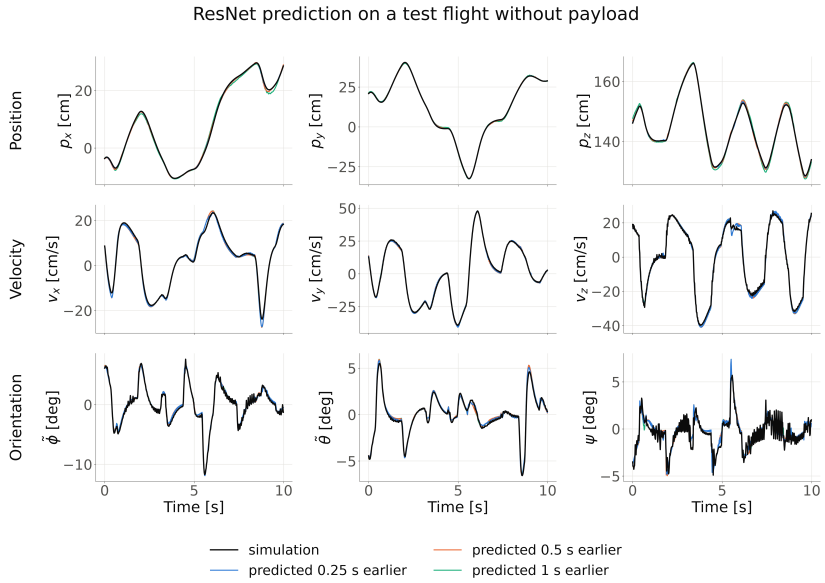


Figure 4.3: ResNet prediction on a test flight without payload: position (top), velocity (middle), and attitude (bottom) over a 10 s stretch. Black: simulation; colored: prediction of the same signal made 0.25, 0.5, and 1 s earlier.

Figure 4.3 shows a test flight without payload. The predictions practically coincide with the simulation even 1 s ahead: over all test flights, the RMS prediction error is below 0.8 cm in position, about 1 cm/s in velocity, and below 0.7° in attitude, even at the longest look-ahead. Only the high-frequency chatter of the yaw angle, caused by the low-level policy, is not reproduced; the model predicts its mean instead.

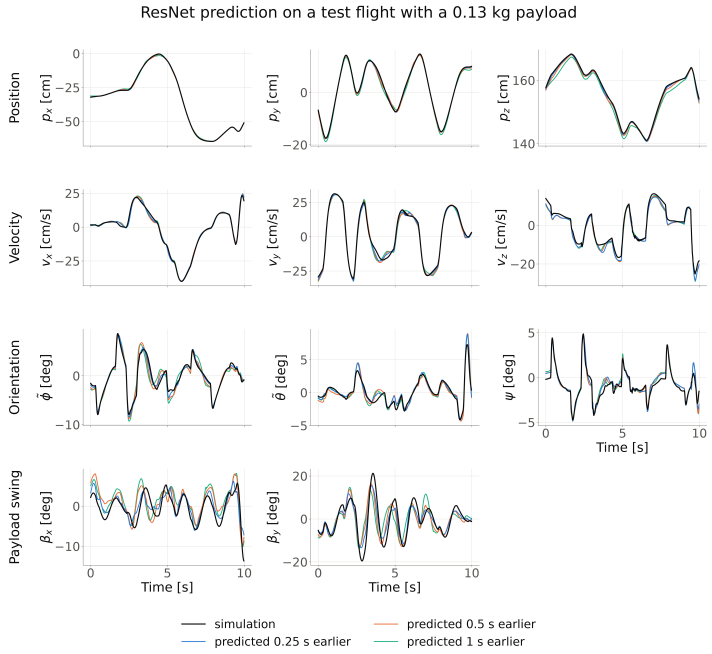


Figure 4.4: ResNet prediction on a test flight with a 0.13 kg payload: position, velocity, attitude, and swing angles of the payload over a 10 s stretch. Black: simulation; colored: prediction of the same signal made 0.25, 0.5, and 1 s earlier.

With the payload, shown in Figure 4.4, the motion of the drone is still predicted accurately, with RMS errors of at most about 2 cm in position, 2.5 cm/s in velocity, and 1° in attitude at 1 s ahead. The swing of the payload is the hardest to predict: its RMS error is about 2° at 0.25 s, and it grows to about 3.8–4.5° at 1 s, as the predicted oscillation accumulates phase and amplitude errors. Since the swing is not penalized in the cost of the MPC, these errors affect the control only through their effect on the motion of the drone.

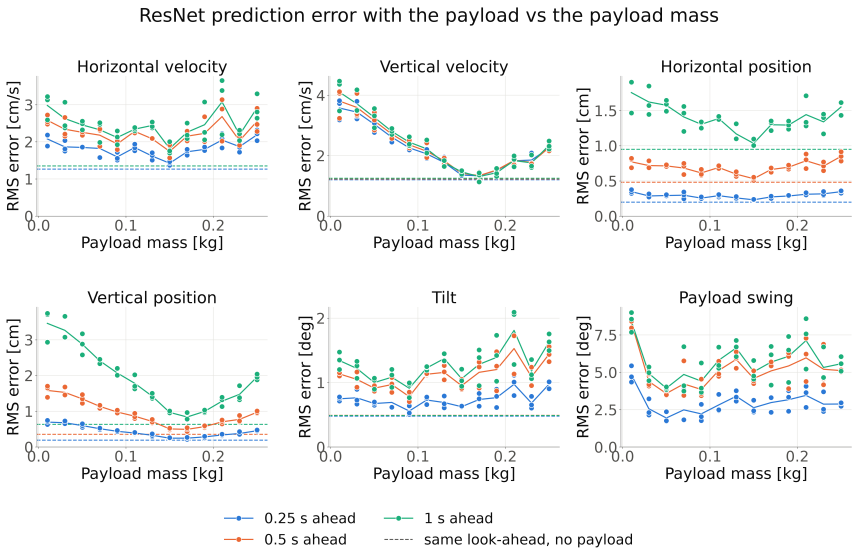


Figure 4.5: RMS prediction error with the payload as a function of the payload mass, 0.25, 0.5, and 1 s ahead. Dots: individual test flights; solid lines: mean over the flights of the same mass; dashed lines: the same error without payload.

Figure 4.5 shows how the prediction error depends on the payload mass. With the payload, the errors are larger than without it for every state, but they remain small compared to the motion of the drone. The vertical velocity and position errors are the largest for the lightest payloads and the smallest around 0.15–0.17 kg. Since the payload mass is not part of the state, the model cannot distinguish between payloads and presumably learns their average effect, which is the most accurate for masses near the middle of the training range. This is a limitation of the model; however, the resulting errors are small compared to the motion of the drone, and since the MPC solves its problem from the measured state at every control step, they are corrected by the feedback. Indeed, the MPC completes the task with payloads of all masses, as shown in the *Comparison* chapter. If needed, the model could be extended with an estimate of the payload mass. The swing error is also the largest for the lightest payload. Overall, the identified model predicts the closed-loop behavior accurately over the horizon of the MPC, both without and with payloads of various masses.

Chapter 5

Safety Filter

As discussed in the *Reinforcement Learning* and *Model Predictive Control* chapters, both methods have their advantages and drawbacks [16]. The RL agent learns the task without an explicit model and computes its commands quickly, but it provides no guarantee that the constraints of the system are satisfied. MPC, on the other hand, handles the constraints explicitly, but its performance depends on the accuracy of the model, and at every control step, it requires rolling out the model over the horizon and computing its Jacobian matrices to solve the optimization problem.

This chapter presents a solution that combines the two methods. The hierarchical agent is extended with an intermediate step: before a command of the high-level policy is applied to the system, a predictive safety filter checks with the identified model whether the command would drive the drone into an unsafe state. If it would not, the command is applied unchanged; otherwise, the filter replaces it with the closest safe command, obtained by solving an MPC problem. The approach follows the predictive safety filter of [24].

The filter is minimally invasive: as long as the commands of the RL agent are safe, they are applied unchanged, so the behavior of the learned policy is preserved, and the filter intervenes only when the predicted state would violate the constraints. The combination is also computationally efficient. The policy network is evaluated quickly, and if the proposed command, followed by the previously planned commands, keeps the predicted state within the constraints, a single rollout of the model is sufficient; an optimization problem is solved only otherwise. At the same time, whenever the filter intervenes, the applied command keeps the state predicted by the identified model within the constraints. Note, however, that this is not a formal safety guarantee for the real system, since the model is only an approximation, and the constraints are softened to keep the problem feasible, as described below.

5.1 Formulation

This section formulates the predictive safety filter. First, it is shown how the filter is placed into the hierarchical agent of Section 3.3, and which signals it receives and produces. Then, the filter is defined as a policy π^{S} that maps the command proposed by the high-level policy to the command applied to the system: it either certifies the proposal or replaces it with the solution of a safety MPC problem. Finally, the constraints that define the safe states are specified, followed by two additional rules for situations that the prediction model does not describe.

5.1.1 Integration into the Hierarchy

The safety filter is placed between the two levels of the hierarchical agent, and neither policy is changed or retrained. The resulting structure is shown in Figure 5.1, which extends the hierarchical agent of Figure 3.2.

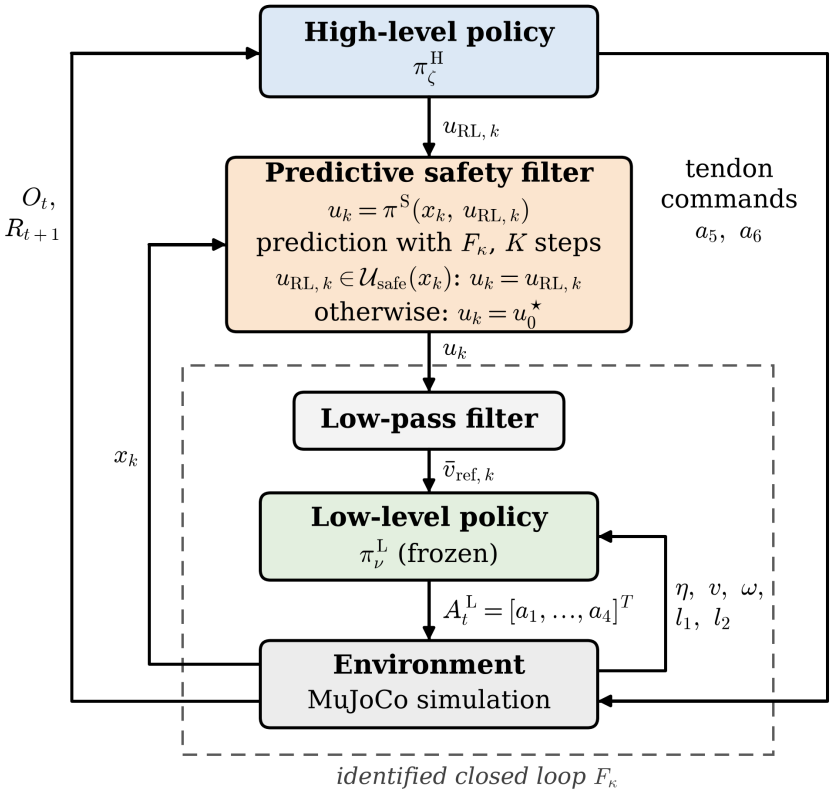


Figure 5.1: Structure of the hierarchical agent with the predictive safety filter π^{S}, following [24]. The filter receives the command $u_{\mathrm{RL},k}$ proposed by the high-level policy and the measured state x_k, and applies either the proposal or the first command $u_0^\star$ of the safety MPC problem. The dashed frame marks the closed loop described by the identified model F_κ.

At each control step k, which corresponds to the time step t of the RL formulation, the signals of Figure 5.1 are the following:

- The high-level policy selects its action $A_t^{\mathrm{H}} \sim \pi_\zeta^{\mathrm{H}}(\cdot \mid O_t^{\mathrm{H}})$ according to (3.14), based on the observation O_t and trained with the reward R_{t+1}. Its reference velocity is the proposed command
$$u_{\mathrm{RL},k} = v_{\mathrm{ref},t}, \tag{5.1}$$
while its tendon commands a_5 and a_6 bypass the filter and are applied unchanged.

- The safety filter maps the proposal and the measured state x_k of (4.1) to the applied command
$$u_k = \pi^{\mathrm{S}}(x_k,\, u_{\mathrm{RL},k})\,, \tag{5.2}$$
where u_k is the command of Section 4.1.1. To decide on u_k, the filter predicts the state over a horizon of K steps. If the proposal belongs to the set $\mathcal{U}_{\mathrm{safe}}(x_k)$ of safe commands, defined in Section 5.1.2, it is applied unchanged; otherwise, the filter applies the first command $u_0^\star$ of the safety MPC problem.

- The applied command is smoothed by the low-pass filter (3.19) in place of $v_{\mathrm{ref},t}$, and the frozen low-level policy π_ν^{L} of (3.15) tracks the filtered reference $\bar{v}_{\mathrm{ref},k}$ with the motor commands A_t^{L}, based on its observation (3.16).

The low-pass filter, the low-level policy, and the drone, marked by the dashed frame in Figure 5.1, form exactly the closed loop identified in Section 4.2. Therefore, the filter predicts the effect of the

commands with the same model as the MPC,

$$x_{j+1} = x_j + F_\kappa(x_j,\, u_j)\,, \qquad x_0 = x_k, \tag{5.3}$$

where κ is the current mode, and the index $j = 0, \ldots, K$ denotes the predicted steps within the horizon.

5.1.2 Safety Filter Problem

The task of the filter is to keep the predicted state within a set of safe states, while changing the proposal as little as possible. The safe states are described by bounds on constrained quantities $y_g(x)$, grouped into constraint groups $g \in \mathcal{G}$:

$$\underline{y}_g \le y_g(x_j) \le \overline{y}_g, \qquad g \in \mathcal{G},\ j = 1, \ldots, K, \tag{5.4}$$

whose groups and bounds are specified in Section 5.1.3. A command is safe if, after applying it, the drone can still be kept within these bounds over the whole horizon. Accordingly, the set of safe commands in the state x_k is

$$\mathcal{U}_{\text{safe}}(x_k) = \{u_0 \ :\ \exists\, u_1, \ldots, u_{K-1} \text{ such that } \|u_j\|_\infty \le u_{\max} \text{ and (5.3), (5.4) hold}\}\,. \tag{5.5}$$

The filter policy π^{S} consists of two steps. First, the proposal is certified:

$$\pi^{\text{S}}(x_k,\, u_{\text{RL},k}) = u_{\text{RL},k}, \qquad \text{if } u_{\text{RL},k} \in \mathcal{U}_{\text{safe}}(x_k). \tag{5.6}$$

To check this condition, the proposal is first followed by the shifted plan of the previous step and predicted with (5.3); if this prediction already satisfies (5.4), the proposal is certified without optimization. Otherwise, a feasible continuation $u_1, \ldots, u_{K-1}$ is searched for with $u_0 = u_{\text{RL},k}$ fixed, and the proposal is certified if one is found.

If the proposal cannot be certified, the filter intervenes, and it applies the first command $u_0^\star$ of the solution of the safety MPC problem

$$\begin{aligned}
\min_{u_0,\ldots,u_{K-1}} \quad & \lambda_0 \left\|u_0 - u_{\text{RL},k}\right\|^2 + \lambda_1 \sum_{j=1}^{K-1} \left\|u_j - u_{\text{RL},k}\right\|^2 \\
& + \lambda_{\Delta u} \sum_{j=0}^{K-1} \left\|u_j - u_{j-1}\right\|^2 && \text{(5.7a)} \\
\text{s.t.} \quad & x_0 = x_k, \qquad x_{j+1} = x_j + F_\kappa(x_j,\, u_j)\,, \qquad j = 0, \ldots, K-1, && \text{(5.7b)} \\
& \underline{y}_g \le y_g(x_j) \le \overline{y}_g, \qquad g \in \mathcal{G},\ j = 1, \ldots, K, && \text{(5.7c)} \\
& \left|\tilde{\phi}_j\right| \le \phi_{\text{hard}}, \quad \left|\tilde{\theta}_j\right| \le \phi_{\text{hard}}, \qquad j = 1, \ldots, K, && \text{(5.7d)} \\
& \left\|u_j\right\|_\infty \le u_{\max}, \qquad j = 0, \ldots, K-1, && \text{(5.7e)}
\end{aligned}$$

i.e.,

$$\pi^{\text{S}}(x_k,\, u_{\text{RL},k}) = u_0^\star, \qquad \text{if } u_{\text{RL},k} \notin \mathcal{U}_{\text{safe}}(x_k), \tag{5.8}$$

where

- the first term of (5.7a) keeps the applied command as close as possible to the proposal, the second term keeps the rest of the plan close to it as well, and the third term, with u_{-1} being the previously applied command, makes the commands smooth,
- (5.7b) is the prediction model (5.3),
- (5.7c) are the constraints (5.4), specified in Section 5.1.3,
- (5.7d) is a hard bound ϕ_{hard} on the tilt angles of (4.1), and
- (5.7e) bounds the commands.

The weight $\lambda_0 \gg \lambda_1$ makes the intervention as small as possible at the current step, while the rest of the plan only has to be feasible. If the hard tilt bound cannot be satisfied, e.g., because the drone is already tilted beyond it, the problem is solved without (5.7d). Both the certification and (5.7) are solved in the same way as the MPC problem of Section 4.1.3, with Gauss–Newton SQP iterations on the linearized model. The parameters of the filter are summarized in Table 5.1.

Parameter	Symbol	Value
Horizon	K	20 steps (0.42 s)
Weight of the first command	λ_0	10
Weight of the continuation	λ_1	0.1
Command change weight	$\lambda_{\Delta u}$	0.5
Command bound	$u_{\max}$	1
Hard tilt bound	ϕ_{hard}	60°
SQP iterations	–	2

Table 5.1: Parameters of the safety filter.

5.1.3 Constraints

The constraints (5.7c) are implemented as soft constraints, i.e., their violation is not forbidden, but it is heavily penalized in the cost, so that the problem remains solvable even if no command can satisfy them exactly, e.g., when the drone is already outside the bounds. The filter uses four constraint groups $\mathcal{G} = \{\text{tilt}, \text{altitude}, \text{speed}, \text{swing}\}$ in (5.4), summarized in Table 5.2: the drone must not tilt too much, approach the ground, fly too fast, or let the payload swing too much. The tilt bound of 15° keeps the drone in the region where the identified model is accurate, while the hard bound $\phi_{\text{hard}} = 60°$ in (5.7d) lies well below the 90° at which the drone flips over. The swing constraint is only used in mode $\kappa = 1$, when the payload is attached. The penalties of the violations set the priorities of the groups, tilt > altitude > speed > swing, so that if not all constraints can be satisfied, the tilt is the last to be violated.

Group g	Constrained quantity $y_g(x)$	Bound
Tilt	$\tilde{\phi},\ \tilde{\theta}$	$\lvert\cdot\rvert \le 15°$
Altitude	p_z	$\ge 0.3\,\text{m}$
Speed	$v_x,\ v_y,\ v_z$	$\lvert\cdot\rvert \le 1\,\text{m/s}$
Swing	$\beta_x,\ \beta_y$ (only in mode $\kappa = 1$)	$\lvert\cdot\rvert \le 45°$

Table 5.2: Constraint groups of the safety filter.

If the measured state already violates a bound, e.g., because the payload already swings more than the limit, an immediate return is impossible. In this case, the bound is relaxed to the current value and has to shrink by a factor of 0.9 per predicted step, so that the state is driven back gradually.

5.1.4 Additional Rules

Two situations are handled by additional rules, since the prediction model does not describe them. First, when the hook touches the payload before it is lifted and the drone starts to tilt by more than 10°, the hook is being levered against the payload, and the drone would flip within about 0.5 s. Since the model contains no contact forces, this is detected from the measured state, and the first command is restricted to a vertical climb until the contact is released and the drone is level again. Second, if the drone stalls, i.e., it moves less than 0.1 m within 1 s while it is farther than 0.25 m from the active waypoint, the MPC of Section 4.1 takes over for 0.5 s and flies toward the waypoint; its commands pass through the same filter.

Chapter 6

Comparison

This chapter compares the three methods that solve the transportation task with the same low-level policy: the RL director, i.e., the high-level policy of Section 3.3.7, the MPC director of the *Model Predictive Control* chapter, and the RL director extended with the safety filter of the *Safety Filter* chapter. The comparison focuses on the question of whether the safety filter is able to save the RL director in the situations in which it fails.

6.1 Evaluation Setup

The methods are evaluated on 300 randomly generated scenarios of the transportation task described in Section 2.3, with a scene similar to Figure 2.6. The task parameters are sampled from the bounds of Table 2.2, which correspond to the final curriculum level. In the following, the payload mass denotes the total mass of the payload, i.e., the disk mass m_{payload} plus the 0.01 kg of the cylinder, which lies between 0.02 and 0.26 kg. Each scenario is defined by a random seed, so all three methods fly exactly the same scenarios. Moreover, the two runs of the RL director with and without the filter differ only through the interventions of the filter. An episode lasts at most 25 s, and its outcome is one of the following:

- *success*: the drone reaches the goal within 0.15 m with the payload, and the payload stays on the hook for 5 s without a crash,
- *stability*: the drone flips over, i.e., its roll or pitch angle exceeds 90°, or it hits the ground,
- *payload*: the payload is lifted but lost again, and
- *unfinished*: none of the above happens within the time limit.

6.2 Results

The results are summarized in Figure 6.1. The RL director completes 283 of the 300 scenarios (94.3 %), the MPC director 299 (99.7 %), and the RL director with the safety filter 293 (97.7 %). Almost all failures of the RL director are stability failures after the payload has been lifted. As the success rate as a function of the payload mass shows, the failures concentrate on the heaviest payloads: for the heaviest fifth of the scenarios, above 0.21 kg, the success rate of the RL director drops to 76.7 %, which the safety filter raises to 90 %, while the MPC director still succeeds in 98.3 % of these scenarios.

The higher success rate of the MPC comes at the cost of speed: the median time to reach the goal is 3.6 s for the RL director, but 5.7 s for the MPC director, since the tilt limit and the penalty on the command changes keep its motion moderate. The safety filter preserves the speed of the RL director, with a median of 3.7 s, since it intervenes in only 2 % of the control steps. The same pattern appears in the maximum tilt of the drone: the median is 26.7° for the RL director, 15.9° for the MPC director, and 20.7° with the safety filter.

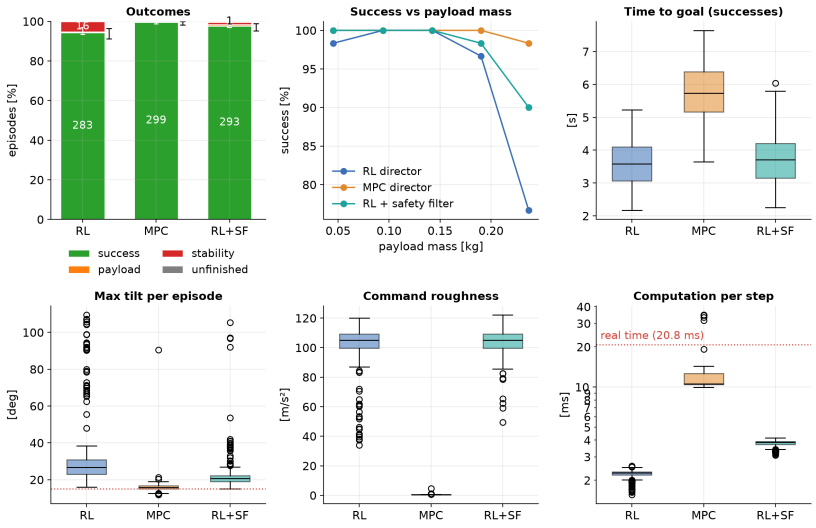


Figure 6.1: Comparison of the RL director (RL), the MPC director (MPC), and the RL director with the safety filter (RL+SF) on the same 300 scenarios. Dotted lines: tilt limit of 15° and control period of 20.8 ms.

The commands of the RL director change rapidly, with a mean rate of change of about 105 $\mathrm{m/s^2}$, while the commands of the MPC director are smooth, since their changes are penalized in its cost. Since the filter changes only a few commands, the roughness of the RL commands remains. In terms of computation, the RL director requires a median of 2.3 ms per control step and the safety filter 3.8 ms, since in most steps a single rollout of the model is sufficient to certify the proposal. The MPC director requires 10.6 ms, several times more, since it solves the optimization problem at every control step. Its implementation could still be accelerated considerably, e.g., by generating the solver with the acados software package [22].

6.3 Rescue of the RL Failures

Since all methods fly the same scenarios, the outcome of the RL director can be compared with the outcome of the other two methods scenario by scenario, as shown in Figure 6.2. The MPC director succeeds in all 17 scenarios in which the RL director fails, and it fails in only one scenario that the RL director completes. The safety filter turns 12 of the 17 failures of the RL director into a success, while it breaks 2 of its 283 successes: in one scenario, the drone does not manage to grasp the payload within the time limit, and in another, it loses the payload.

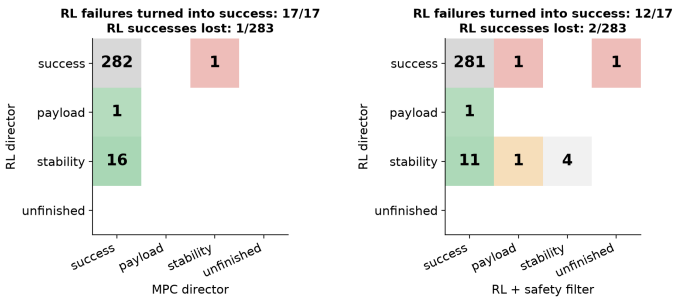


Figure 6.2: Outcome of the RL director (rows) and of the MPC director (left) or the RL director with the safety filter (right) on the same scenario (columns).

The 5 failures that the filter does not prevent all occur with payloads heavier than 0.24 kg, where the filter is active in 17–35 % of the control steps. Since the filter corrects the proposals only as much as the constraints require, and the identified model is the least accurate with the heavy, strongly swinging payload, this is not always sufficient.

A rescued scenario with a 0.25 kg payload is shown in Figures 6.3 and 6.4. The RL director commands the full velocity toward the goal while the hook still touches the payload, so it drags the payload off the stand, and the payload starts to swing strongly. The drone overshoots, and the swinging payload pulls it over: 4.1 s after the start, about 1 m from the goal, its tilt exceeds 90°. With the safety filter, the filter detects the contact of the hook with the payload and replaces the proposal with a vertical climb for 0.3 s, until the contact is released. Afterwards, the proposals are applied almost unchanged, and the drone reaches the goal after 4.0 s and holds the payload. The MPC director flies more slowly, and it reaches the goal after 5.8 s.

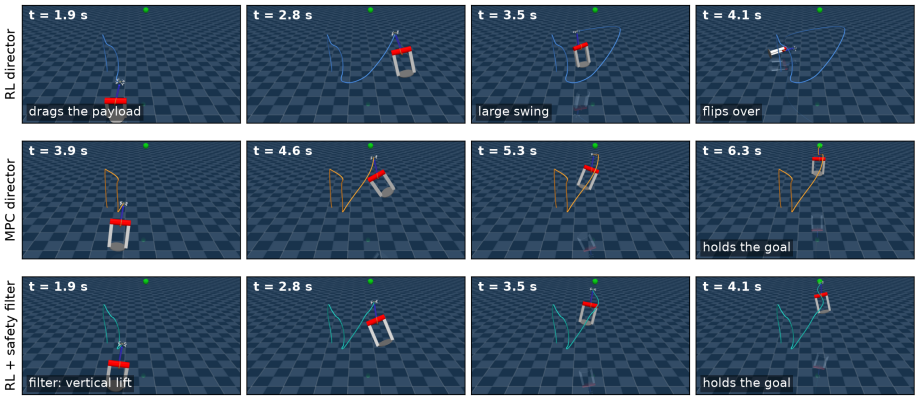


Figure 6.3: Snapshots of the rescued scenario with a 0.25 kg payload, flown by the three methods. The line shows the path of the drone, and the green sphere is the goal.

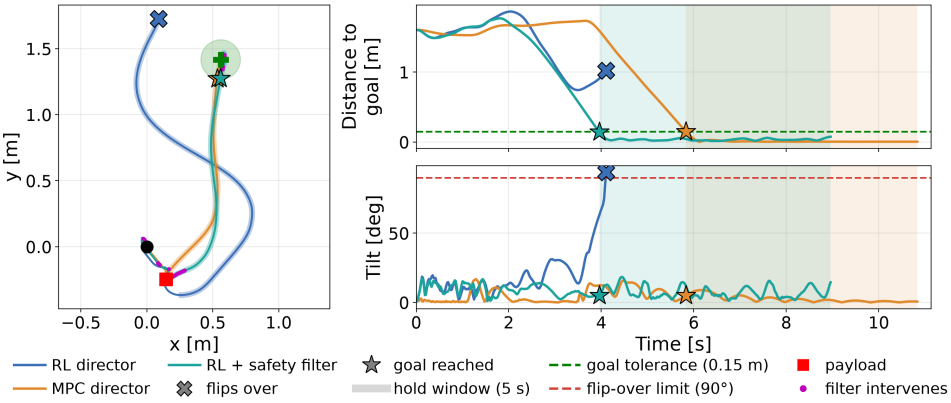


Figure 6.4: The scenario of Figure 6.3 with all three methods: path from above, thick where the payload is carried (left), and the distance to the goal and the tilt (right). Success: within 0.15 m of the goal with the payload (star), which is kept for 5 s (shaded); stability failure: tilt above 90° (cross). Each curve ends when the outcome is decided.

Overall, the MPC director is the most reliable, but also the slowest and the most computationally demanding method. The RL director is the fastest and requires the least computation, but it fails in about 6 % of the scenarios, mainly with heavy payloads. The safety filter removes most of these failures, while it keeps the speed of the RL director at a much lower computational cost than the MPC. The technical details are given in Appendix A.2.

Chapter 7

Conclusion and Future Work

This thesis showed how reinforcement learning and model predictive control can be applied to a drone equipped with an adaptive hook, a manipulator inspired by spiral robots, which has to grasp a payload with unknown mass and size and transport it to a goal position. The RL approach, a hierarchical agent trained with curriculum learning, solves the task without a model of the system, while the MPC relies on a ResNet model of the closed loop of the drone and the low-level policy, identified from simulation data. The two methods were compared on the same 300 scenarios, both in terms of safety, i.e., how often the drone flips over or loses the payload, and in terms of performance, i.e., speed and computational cost. The RL agent proved to be the fastest and computationally cheapest, but it failed in about 6 % of the scenarios, mainly with heavy payloads, whereas the MPC was the most reliable, but also slower and computationally more demanding.

Building on these results, a hybrid method was proposed, in which a predictive safety filter, based on the same identified model, supervises the commands of the RL agent and intervenes only when they would drive the drone into an unsafe state. The filter raised the success rate of the RL agent from 94.3 % to 97.7 %, while it intervened in only 2 % of the control steps, so the speed of the RL agent was preserved. These results in simulation are promising.

In the future, the proposed methods are planned to be tested on the real system. Further possible improvements are the extension of the identified model with an estimate of the payload mass, and a faster implementation of the MPC and the safety filter, e.g., with the acados software package.

Acknowledgements

This work would never have been completed without the supportive environment that encouraged me throughout it. First of all, I would like to thank my girlfriend, who stood by me and helped me through the more difficult periods, and my family, who supported me without question or hesitation.

I would also like to thank Bence Szinyéri, who provided valuable advice and the framework for this work at the Budapest University of Technology and Economics, and Péter Antal, researcher at HUN-REN SZTAKI, whose insights helped this work move forward.

Bibliography

[1] Joel A E Andersson, Joris Gillis, Greg Horn, James B Rawlings, and Moritz Diehl. CasADi – A software framework for nonlinear optimization and optimal control. *Mathematical Programming Computation*, 11(1):1–36, 2019. DOI: 10.1007/s12532-018-0139-4.

[2] Péter Antal, Tamás Péni, and Roland Tóth. Modelling, identification and geometric control of autonomous quadcopters for agile maneuvering. *Repüléstudományi Közlemények*, 35(1): 141–160, 2023. DOI: 10.32560/rk.2023.1.11.

[3] Péter Antal, Andrea Carron, Melanie Zeilinger, Roland Tóth, and Tamás Péni. Robust adaptive predictive control for hook-based aerial transportation between moving platforms. *IEEE Robotics and Automation Letters*, 11(11):12352–12359, 2026. DOI: 10.1109/LRA.2026.3730216.

[4] Daniel Arnström and Daniel Axehill. A dual active-set solver for embedded quadratic programming. *IEEE Transactions on Automatic Control*, pages 1–1, 2022. DOI: 10.1109/TAC.2022.3176430.

[5] Pierre Clément Blaud, Philippe Chevrel, Fabien Claveau, Pierrick Haurant, and Anthony Mouraud. Resnet and polynet based identification and (mpc) control of dynamical systems: A promising way. *IEEE Access*, 11:20657–20672, 2023. DOI: 10.1109/ACCESS.2022.3196920.

[6] Stephen Boyd and Lieven Vandenberghe. *Convex Optimization*. Cambridge University Press, 2004.

[7] Joshua Fishman and Luca Carlone. Control and trajectory optimization for soft aerial manipulation. In *2021 IEEE Aerospace Conference (50100)*, pages 1–17, 2021. DOI: 10.1109/AERO50100.2021.9438502.

[8] David Kiessling. *Algorithmic Advances in Feasible-Iterate and SQP Methods for Direct Optimal Control*. PhD thesis, KU Leuven, 2025.

[9] Martin Klissarov, Akhil Bagaria, Ziyan Luo, George Konidaris, Doina Precup, and Marlos C. Machado. Discovering temporal structure: An overview of hierarchical reinforcement learning, 2026. URL https://arxiv.org/abs/2506.14045.

[10] Hughes La Vigne, Guillaume Charron, Julien Rachiele-Tremblay, David Rancourt, Ben Nyberg, and Alexis Lussier Desbiens. Collecting critically endangered cliff plants using a drone-based sampling manipulator. *Scientific Reports*, 12(1):14827, 2022. ISSN 2045-2322. DOI: 10.1038/s41598-022-17679-x. URL https://doi.org/10.1038/s41598-022-17679-x.

[11] Chen Liang and Daniel Rakita. Robust and efficient mujoco-based model predictive control via web of affine spaces derivatives, 2026. URL https://arxiv.org/abs/2512.21109.

[12] Ezeldin N. Moustafa, Mitsuhiro Kamezaki, Ahmed Khalifa, Shota Miyake, Tamon Miyake, and Shigeki Sugano. Development of a two-dof three-parallelogram aerial manipulator for pipeline crack repair using a two-tool end-effector and onboard sensor-based localization. In *2025 IEEE/ASME International Conference on Advanced Intelligent Mechatronics (AIM)*, pages 1–7, 2025. DOI: 10.1109/AIM64088.2025.11175832.

[13] Sanmit Narvekar, Bei Peng, Matteo Leonetti, Jivko Sinapov, Matthew E. Taylor, and Peter Stone. Curriculum learning for reinforcement learning domains: A framework and survey. *CoRR*, abs/2003.04960, 2020. DOI: 10.48550/ARXIV.2003.04960. URL https://doi.org/10.48550/arXiv.2003.04960.

[14] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Kopf, Edward Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. Pytorch: An imperative style, high-performance deep learning library. In H. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché-Buc, E. Fox, and R. Garnett, editors, *Advances in Neural Information Processing Systems 32*, pages 8024–8035. Curran Associates, Inc., 2019. URL http://papers.neurips.cc/paper/9015-pytorch-an-imperative-style-high-performance-deep-learning-library.pdf.

[15] Antonin Raffin, Ashley Hill, Adam Gleave, Anssi Kanervisto, Maximilian Ernestus, and Noah Dormann. Stable-baselines3: Reliable reinforcement learning implementations. *Journal of Machine Learning Research*, 22(268):1–8, 2021. URL https://jmlr.org/papers/v22/20-1364.html.

[16] Rudolf Reiter, Jasper Hoffmann, Dirk Reinhardt, Florian Messerer, Katrin Baumgärtner, Shambhuraj Sawant, Joschka Bödecker, Moritz Diehl, and Sebastien Gros. Synthesis of model predictive control and reinforcement learning: Survey and classification. *Annual Reviews in Control*, 61:101045, 2026. ISSN 1367-5788. DOI: https://doi.org/10.1016/j.arcontrol.2026.101045. URL https://www.sciencedirect.com/science/article/pii/S1367578826000015.

[17] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms, 2017. URL https://arxiv.org/abs/1707.06347.

[18] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. Adaptive Computation and Machine Learning. MIT Press, Cambridge, MA, 2nd edition, 2018. ISBN 978-0-262-03924-6. URL http://incompleteideas.net/book/the-book-2nd.html.

[19] Manan Tayal. Mujoco-drones-gym: A gpu-accelerated multi-drone simulator for control and reinforcement learning, 2026. URL https://arxiv.org/abs/2606.08039.

[20] Emanuel Todorov, Tom Erez, and Yuval Tassa. Mujoco: A physics engine for model-based control. In *2012 IEEE/RSJ International Conference on Intelligent Robots and Systems*, pages 5026–5033. IEEE, 2012. DOI: 10.1109/IROS.2012.6386109.

[21] Mark Towers, Ariel Kwiatkowski, John Balis, Gianluca De Cola, Tristan Deleu, Manuel Goulão, Kallinteris Andreas, Markus Krimmel, Arjun KG, Rodrigo Perez-Vicente, J Terry, Andrea Pierré, Sander Schulhoff, Jun Jet Tai, Hannah Tan, and Omar G. Younis. Gymnasium: A standard interface for reinforcement learning environments. In D. Belgrave, C. Zhang, H. Lin, R. Pascanu, P. Koniusz, M. Ghassemi, and N. Chen, editors, *Advances in Neural Information Processing Systems*, volume 38, Main Conference. Curran Associates, Inc., 2025. DOI: 10.52202/085713-4916. URL https://proceedings.neurips.cc/paper_files/paper/2025/file/d7ff1795e8527f6443371c3933bdb52b-Paper-Datasets_and_Benchmarks_Track.pdf.

[22] Robin Verschueren, Gianluca Frison, Dimitris Kouzoupis, Jonathan Frey, Niels van Duijkeren, Andrea Zanelli, Branimir Novoselnik, Thivaharan Albin, Rien Quirynen, and Moritz Diehl. acados – a modular open-source framework for fast embedded optimal control. *Mathematical Programming Computation*, 2021.

[23] B. V. Vidyadhara, Lima Agnel Tony, Mohitvishnu S. Gadde, Shuvrangshu Jana, V. P. Varun, Aashay Anil Bhise, Suresh Sundaram, and Debasish Ghose. Design and integration of a drone based passive manipulator for capturing flying targets. *Robotica*, 40(7):2349–2364, 2022. DOI: 10.1017/S0263574721001673.

[24] Kim Peter Wabersich and Melanie N. Zeilinger. A predictive safety filter for learning-based control of constrained nonlinear dynamical systems. *Automatica*, 129:109597, 2021. ISSN 0005-1098. DOI: https://doi.org/10.1016/j.automatica.2021.109597. URL https://www.sciencedirect.com/science/article/pii/S0005109821001175.

[25] Zhanchi Wang, Nikolaos M. Freris, and Xi Wei. Spirobs: Logarithmic spiral-shaped robots for versatile grasping across scales. *Device*, 3(4):100646, 2025. ISSN 2666-9986. DOI: https://doi.org/10.1016/j.device.2024.100646. URL https://www.sciencedirect.com/science/article/pii/S2666998624006033.

[26] Chao Zeng, Zhan Li, Yipeng Yang, Cunjia Liu, and Chenguang Yang. Learning-based methods for aerial manipulation: A focused review. *IEEE/ASME Transactions on Mechatronics*, 31 (2):2270–2287, 2026. DOI: 10.1109/TMECH.2025.3625915.

[27] Wen-Shuai Zhao, Jorge Peña Queralta, and Tomi Westerlund. Sim-to-real transfer in deep reinforcement learning for robotics: a survey. *2020 IEEE Symposium Series on Computational Intelligence (SSCI)*, pages 737–744, 2020. URL https://api.semanticscholar.org/CorpusID:221971078.

Appendix

A.1 Hyperparameters

All policies of the *Reinforcement Learning* chapter are trained with the PPO implementation of Stable-Baselines3 [15], with the hyperparameters of Table A.1. The entropy coefficient is relatively high, which encourages exploration: the training becomes more aggressive, i.e., the return fluctuates more, but it remains manageable. The training parameters of the ResNet models of Section 4.2.2 are given in Table A.2.

Parameter	Value
Actor and critic networks	separate, 2 hidden layers of 64 units, tanh
Learning rate	$3 \cdot 10^{-4}$
Discount factor γ	0.99
GAE parameter λ	0.95
Clip range	0.2
Entropy coefficient	0.01
Value function coefficient	0.5
Steps per environment and update	2048
Minibatch size / epochs per update	64 / 10
Gradient norm clipping	0.5

Table A.1: Hyperparameters of PPO.

Parameter	Value
Hidden units / residual blocks	128 / 2
Final rollout horizon H	24 steps (0.5 s)
Optimizer	Adam, learning rate 10^{-3}, cosine annealing
Epochs / batch size	150 / 256 windows
Gradient norm clipping	1

Table A.2: Training parameters of the ResNet model.

A.2 Technical Details

The code of this thesis, including the simulation environments, the training of the policies, the identification, the MPC, the safety filter, and the scripts of the figures, is available in the repository `https://github.com/Eper00/Adaptive-hook-drone`, which is a fork of the MuJoCo-Drones-Gym simulator [19]. All paths below are relative to the root of this repository.

Software

The simulation is implemented in MuJoCo 3.10 [20], with Gymnasium 1.3 [21] environments, in Python 3.12. The policies are trained with the PPO implementation of Stable-Baselines3 2.9 [15],

and the ResNet models with PyTorch 2.13 [14]. The MPC and the safety filter use CasADi 3.8 [1] for the derivatives of the model, and the quadratic programs of their SQP iterations are solved with the DAQP solver [4] through CasADi.

Hardware

All computations were carried out on a personal computer with an Intel Core i5-14600KF processor. The policies and the ResNet models were trained on the CPU. In the evaluation of Chapter 6, the episodes ran in parallel processes with one thread each, and the computation times were measured in these processes.

Use of AI Tools

The AI assistant Claude (Anthropic), used through Claude Code, assisted in this work with writing code, i.e., parts of the analysis and plotting scripts listed below and the generation of the figures, and with the preparation of this document, i.e., the formatting of the LaTeX source, the drafting of parts of the text, and its language editing. All results were produced with the author's code and simulations, and all content was reviewed by the author, who takes full responsibility for it.

Results

The trained models and the outputs of the scripts are stored in the `results` directory, summarized in Table A.3. The figures of the thesis are written to `results/thesis_figures`.

Directory	Content
`final/`	final policies: low-level, high-level, and flat agents
`resnet/`	ResNet models, their training data and training log
`seeds/`	training runs on three random seeds
`analysis/identification/`	validation of the identification
`analysis/methods/`	300 episodes of each method (`episodes.pkl`)
`analysis/scenario/seed_32/`	the scenario of Figures 6.3 and 6.4
`analysis/seeds/`	training curves of the individual seed groups
`thesis_figures/`	figures of the thesis

Table A.3: Content of the `results` directory.

Scripts

The main scripts and the results they produce are summarized in Table A.4.

Script	Purpose
`utilities/learn.py`	training of the PPO policies
`utilities/train_seeds.sh`	training on three random seeds
`utilities/plot_seeds.py`	Figures 3.1 and 3.8 (`-overlay`)
`utilities/plot_curriculum_training.py`	Figures 3.3 and 3.7
`utilities/analyse.py`	Figures 3.4–3.6 (`-test velocity`); single scenarios (`-test scenario`)
`multi_drone_mujoco/envs/resnet_mpc.py`	ResNet model, its training, and the MPC
`utilities/resnet_training_curve.py`	Figure 4.2
`utilities/test_resnet.py`	Figures 4.3–4.5
`multi_drone_mujoco/envs/predictive_safety_filter.py`	predictive safety filter
`utilities/compare_methods.py`	evaluation of the three methods
`utilities/plot_comparison_thesis.py`	Figures 6.1 and 6.2
`utilities/render_rescue_scene.py`	Figure 6.3
`utilities/plot_scenario_thesis.py`	Figure 6.4
`utilities/render_pictures.py`	Figures 2.5, 2.6, and 2.7

Table A.4: Main scripts of the repository.